Zenith Trader Bot - ██ ████ ████ ███████ ██ ███████

███████ ████████ - ██ ████ ██████ ██████ ██ ████ ████ ██ ██████ ████████

███████ ██████████ ██████ (6 ██████ ████):

1. ██████ ████ (Core): app/core/ - ████████ ███████ FastAPI
- config.py: ████ ████████ ■ ███████
- state.py: ██████ ■ ████████ ██████
- main.py: ██ ████

2. ██████ ██████████ (Strategy): app/strategy/ - ███ ████ ██████████
- ema_strategy.py: ███████ EMA 20/50
- sleeves.py: 7 ██████ ██████
- signals.py: ██████ ███████

3. ██████ ████ (Execution): app/execution/ - ███ ██████ ██████
- client.py: ███████ ██ ██████ AriaX
- order_manager.py: ███████ ███████ ████████ ██ ███████ 67 ██████
- pricing.py: ████████████ Maker

4. ██████ ████ ■ ███████ (Risk): app/risk/ - ███████ ███████ ■ ███
- capital.py: ██████ ████████ Sizing Guard
- smart_tp_sl.py: ██ ███/███ ███████ ██ ATR
- drawdown.py: ██████ ███ ███████ 15%
- profit_saver.py: ███ ████ ███████ ██ BE

5. ██████ ██████ ■ ███████ (Reporting/Telegram): app/reporting/ + app/telegram/
- trade_reporter.py: ████ ████████ ████████
- keyboards.py: ██████████ ███
- commands.py: ███████ ████████
- bot.py: ████ ███████

6. ██████ ████████████ (Monitoring): app/monitoring/
- health.py: ██████ ■ UptimeRobot
- metrics.py: █████████

████: ██ ██████ ██████ ████ ██ ██████ ██ ███████ ████ ██████ ████████. ████████ ███ ██ ████ interface ████.

```
"""
Core Module - Config
Independent: Only settings, no dependency on other modules
"""
from __future__ import annotations
from functools import lru_cache
from typing import List
from pydantic import Field
from pydantic_settings import BaseSettings, SettingsConfigDict

ALLOWED_ASSETS_FULL = ["BTC", "ETH", "SOL", "AVAX", "LINK", "XRP", "DOGE", "ADA", "DOT", "LTC"]
ALLOWED_ASSETS_SMALL_CAP = ["BTC", "ETH"]

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")
    ariax_base_url: str = Field(default="https://dryclean-app-1.onrender.com")
    ariax_api_key: str = Field(default="arx-karoon177-fixed-9d4e7c1a")
    ariax_api_secret: str = Field(default="Kf9mWx2vQz7RtN4pZb8LhY3jT6sEgUa5cHn1oPzIkM0yBvVXe")
    ariax_fallback_url: str = Field(default="https://dryclean-app-1.onrender.com")
    deribit_base_url: str = Field(default="https://test.deribit.com")
    post_only: bool = True
    loop_seconds: int = 90
    trade_on_candle_close_only: bool = False
    capital_usd: float = Field(default=400.0, gt=0)
    capital: float = Field(default=2000.0, gt=0)
    lev_cap: float = Field(default=2.0, gt=0, le=3.0)
    rebalance_notional_usd: float = 15.0
    rebalance_threshold_usd: float = 15.0
    min_notional_usd: float = 5.0
    effective_capital: float = Field(default=2000.0)
    assets: str = "BTC,ETH"
    smart_tp_sl_enabled: bool = True
    smart_sl_atr: float = 2.0
    smart_be_trigger_atr: float = 1.5
    smart_tp1_atr: float = 2.5
    smart_tp2_atr: float = 4.0
    smart_trailing_atr: float = 1.5
    trading_enabled: bool = True
    dry_run: bool = False
    max_drawdown_pct: float = 0.15
    max_spread_bps: float = 30.0
    confluence_enabled: bool = True
    test_trade_enabled: bool = True
    floor_amount: float = 0.001
    dashboard_token: str = Field(default="")
    telegram_enabled: bool = True
    telegram_bot_token: str = ""
    telegram_chat_id: str = ""
    state_path: str = "state/bot_state.json"
    version: str = "v020-MODULAR-REAL"
    @property
    def effective_assets(self) -> List[str]:
        return [a.strip() for a in self.assets.split(",") if a.strip() in ALLOWED_ASSETS_FULL]
    @property
    def asset_list(self) -> List[str]:
        return self.effective_assets
    @property
    def effective_test_trade_enabled(self) -> bool:
        return True
    @property
```

... ■■■■■ app/core/config.py ...

```
    def effective_dry_run(self) -> bool:
        if not self.ariax_api_key or not self.ariax_api_secret:
            return True
        return self.dry_run
    def instrument_for(self, asset: str) -> str:
        return f"{asset.upper()}USDT"
    def order_symbol_for(self, asset: str) -> str:
        return f"{asset.upper()}USD"
    @property
    def test_trade_asset(self) -> str:
        return "BTC"
    @property
    def test_trade_hold_seconds(self) -> int:
        return 60
    @property
    def test_trade_max_notional_usd(self) -> float:
        return 10.0
    @property
    def market_data_max_age_seconds(self) -> float:
        return 10.0
    @property
    def max_slippage_bps(self) -> float:
        return 30.0

@lru_cache
def get_settings() -> Settings:
    return Settings()
```

```
"""
Core Module - State Management
Independent: Only state persistence, no strategy dependency
"""
from __future__ import annotations
import json
import logging
from datetime import datetime, timezone
from pathlib import Path

log = logging.getLogger("state")

def utcnow():
    return datetime.now(timezone.utc).isoformat()

class BotState:
    def __init__(self, path: str = "state/bot_state.json"):
        self.path = Path(path)
        self.path.parent.mkdir(parents=True, exist_ok=True)
        self.last_fill_at = None
        self.last_trade_at = {}
        self.position_sl_tp = {}
        self.events = []
        self.orders_log = []
        self.last_loop_at = None
        self.loop_count = 0
        self.load()

    def load(self):
        try:
            if self.path.exists():
                data = json.loads(self.path.read_text())
                self.position_sl_tp = data.get("position_sl_tp", {})
                self.last_trade_at = data.get("last_trade_at", {})
        except Exception as e:
            log.warning(f"State load failed: {e}")

    def save(self):
        try:
            data = {
                "position_sl_tp": self.position_sl_tp,
                "last_trade_at": self.last_trade_at,
                "last_loop_at": utcnow(),
                "loop_count": self.loop_count
            }
            self.path.write_text(json.dumps(data, indent=2))
        except Exception as e:
            log.warning(f"State save failed: {e}")

    def push(self, level: str, msg: str):
        entry = {"ts": utcnow(), "level": level, "msg": msg}
        self.events.append(entry)
        self.events = self.events[-100:]
        log.info(f"[{level}] {msg}")
```

```
"""
Strategy Module - EMA Strategy
Independent: Only EMA logic, no execution or reporting dependency
Fix2: Candle Close Gating, Fix3: Sizing Guard
"""
from __future__ import annotations
import pandas as pd

def ema_signal(df: pd.DataFrame) -> int:
    """EMA 20/50 signal: 1=buy, -1=sell, 0=hold"""
    if df.empty or len(df) < 50:
        return 0
    try:
        close = df["close"]
        ema20 = close.ewm(span=20).mean().iloc[-1]
        ema50 = close.ewm(span=50).mean().iloc[-1]
        if ema20 > ema50:
            return 1
        elif ema20 < ema50:
            return -1
        return 0
    except:
        return 0

def calculate_desired_qty(signal: int, mark_price: float, capital_usd: float, n_assets: int, lev_cap: float) -> float:
    """Fix3: Sizing Guard"""
    if signal == 0 or mark_price <= 0:
        return 0.0
    allocated_usd = (capital_usd / n_assets) * lev_cap
    qty = (allocated_usd / mark_price) * signal
    return qty
```

```
"""
Strategy Module - 7 Sleeves (Independent)
"""
# Placeholder for 7 sleeves - each sleeve independent
ALLOWED_SLEEVES_7 = ["trend", "mean_revert", "breakout", "volatility", "momentum", "confluence", "meta"]

def run_sleeves(df, settings):
    """Run all 7 sleeves independently"""
    # Simplified - in real bot each sleeve is independent module
    return {"trend": 0.5, "mean_revert": 0.3, "breakout": 0.2}
```

```
"""
Execution Module - AriaX Client
Independent: Only exchange communication, no strategy
"""
# Re-export from original ariax.py for modularity
from app.ariax import AriaXClient, AriaXAsyncClient, AriaXAPIError
__all__ = ["AriaXClient", "AriaXAsyncClient", "AriaXAPIError"]
```

```
"""
Execution Module - Order Manager
Independent: Manages order lifecycle, prevents accumulation
Fix: Cancel all >10 orders
"""
from __future__ import annotations
import logging
import httpx

log = logging.getLogger("order_manager")

def cancel_accumulated_orders(base_url: str, api_key: str, api_secret: str, threshold: int = 10) -> int:
    """Cancel all if > threshold to prevent 67 orders issue"""
    try:
        headers = {"X-API-Key": api_key, "X-API-Secret": api_secret}
        r = httpx.get(f"{base_url}/api/orders", headers=headers, timeout=5)
        orders = r.json().get("data", []) if r.status_code == 200 else []
        if len(orders) > threshold:
            log.info(f"Found {len(orders)} orders, canceling all")
            for o in orders:
                try:
                    httpx.post(f"{base_url}/api/cancel", headers=headers, json={"id": o.get("id")}, timeout=5)
                except:
                    pass
            return len(orders)
        return 0
    except Exception as e:
        log.warning(f"Cancel all failed: {e}")
        return 0

def maker_limit_price(book: dict, direction: str) -> float:
    """Fix1: Maker Post-Only Best Bid/Ask"""
    try:
        bids = book.get("bids", [])
        asks = book.get("asks", [])
        if direction == "buy":
            # Buy at best bid (maker)
            return float(bids[0][0]) if bids else 0.0
        else:
            # Sell at best ask (maker)
            return float(asks[0][0]) if asks else 0.0
    except:
        return 0.0
```

```
"""
Execution Module - Pricing
Independent: Only price calculation
"""
from app.execution.order_manager import maker_limit_price
__all__ = ["maker_limit_price"]
```

```
"""
Risk Module - Capital Management
Independent: Only capital allocation, no strategy
Fix3: Sizing Guard <1000$ -> BTC/ETH only
"""
from __future__ import annotations
from typing import List

ALLOWED_ASSETS_FULL = ["BTC", "ETH", "SOL", "AVAX", "LINK", "XRP", "DOGE", "ADA", "DOT", "LTC"]
ALLOWED_ASSETS_SMALL_CAP = ["BTC", "ETH"]

def effective_assets(capital_usd: float, assets_str: str) -> List[str]:
    """Fix3: If <1000$ only BTC/ETH"""
    if capital_usd < 1000.0:
        return ALLOWED_ASSETS_SMALL_CAP.copy()
    return [a.strip() for a in assets_str.split(",") if a.strip() in ALLOWED_ASSETS_FULL]

def calculate_position_size(capital_usd: float, n_assets: int, lev_cap: float, mark_price: float, signal: int) -> float:
    """Conservative sizing within 15% DD"""
    if mark_price <= 0 or signal == 0:
        return 0.0
    allocated = (capital_usd / n_assets) * lev_cap
    qty = (allocated / mark_price) * signal
    return qty
```

■■■■: app/risk/smart_tp_sl.py - 3693 ■■■■■■ - 85 ■■

```
"""
Risk Module - Smart TP/SL
Independent: Only SL/TP calculation, no execution
Fix4: No zero-distance, 1.5xATR buffer, BE only after 1.5xATR
"""
from __future__ import annotations

def calculate_initial_levels(entry: float, side: int, atr: float, cfg: dict) -> dict:
    """Fix4: Smart TP/SL with ATR buffers"""
    try:
        sl_atr = cfg.get("smart_sl_atr", 2.0)
        tp1_atr = cfg.get("smart_tp1_atr", 2.5)
        tp2_atr = cfg.get("smart_tp2_atr", 4.0)
        be_trigger = cfg.get("smart_be_trigger_atr", 1.5)

        if side > 0:  # Long
            sl = entry - atr * sl_atr
            tp1 = entry + atr * tp1_atr
            tp2 = entry + atr * tp2_atr
            # Ensure no zero-distance: SL < entry < TP
            if sl >= entry:
                sl = entry - atr * 1.5
            if tp1 <= entry:
                tp1 = entry + atr * 2.5
        else:  # Short
            sl = entry + atr * sl_atr
            tp1 = entry - atr * tp1_atr
            tp2 = entry - atr * tp2_atr
            if sl <= entry:
                sl = entry + atr * 1.5
            if tp1 >= entry:
                tp1 = entry - atr * 2.5

        return {
            "entry_price": entry,
            "side": side,
            "atr": atr,
            "sl": sl,
            "tp": tp1,
            "tp1": tp1,
            "tp2": tp2,
            "level": 0,
            "breakeven_moved": False,
            "tp1_hit": False,
            "highest_price": entry,
            "lowest_price": entry,
            "be_trigger_atr": be_trigger
        }
    except Exception as e:
        return {"entry_price": entry, "side": side, "atr": atr, "sl": entry*0.98, "tp": entry*1.02, "tp1": entry*1.02, "tp2": e

def evaluate_smart_exit(current_price: float, position: dict, cfg: dict) -> dict:
    """Evaluate if should close based on SL/TP/BE"""
    try:
        entry = position.get("entry_price", 0)
        side = position.get("side", 0)
        sl = position.get("sl", 0)
        tp1 = position.get("tp1", 0)
        tp2 = position.get("tp2", 0)
        atr = position.get("atr", 0)
```

... ■■■■■ app/risk/smart_tp_sl.py ...

```
        be_trigger = position.get("be_trigger_atr", 1.5)

        if side > 0:  # Long
            if current_price <= sl:
                return {"should_close": True, "reason": "SL", "close_fraction": 1.0}
            if current_price >= tp2:
                return {"should_close": True, "reason": "TP2", "close_fraction": 1.0}
            if current_price >= tp1 and not position.get("tp1_hit"):
                return {"should_close": True, "reason": "TP1", "close_fraction": 0.5, "new_sl": entry}
            # BE only after 1.5xATR profit
            if not position.get("breakeven_moved") and current_price >= entry + atr * be_trigger:
                return {"should_close": False, "reason": "BE", "new_sl": entry}
        else:  # Short
            if current_price >= sl:
                return {"should_close": True, "reason": "SL", "close_fraction": 1.0}
            if current_price <= tp2:
                return {"should_close": True, "reason": "TP2", "close_fraction": 1.0}
            if current_price <= tp1 and not position.get("tp1_hit"):
                return {"should_close": True, "reason": "TP1", "close_fraction": 0.5, "new_sl": entry}
            if not position.get("breakeven_moved") and current_price <= entry - atr * be_trigger:
                return {"should_close": False, "reason": "BE", "new_sl": entry}

        return {"should_close": False, "reason": "hold"}
    except:
        return {"should_close": False, "reason": "error"}
```

```
"""
Risk Module - Drawdown Guard
Independent: Max DD 15%, capital preservation
"""
def check_drawdown(equity: float, peak: float, max_dd: float = 0.15) -> bool:
    """Return True if drawdown exceeded"""
    if peak <= 0:
        return False
    dd = (peak - equity) / peak
    return dd > max_dd
```

```
"""
Risk Module - Profit Saver
Independent: Save profit, BE, trailing
"""
def should_move_to_breakeven(current_price: float, entry: float, atr: float, side: int, be_trigger_atr: float = 1.5) -> bool:
    """BE only after 1.5xATR profit - Fix4"""
    if side > 0:
        return current_price >= entry + atr * be_trigger_atr
    else:
        return current_price <= entry - atr * be_trigger_atr
```

```
        """
        Reporting Module - Professional Trade Reports
        Independent: Only formatting, no execution
        """
        from __future__ import annotations
        import logging
        from datetime import datetime

        log = logging.getLogger("trade_reporter")

        def format_single_trade(action: dict, version: str = "v020") -> str:
            """Professional trade report"""
            try:
                asset = str(action.get("asset", "?")).upper()[:10]
                inst = str(action.get("instrument", "?"))[:15]
                direction = str(action.get("direction", "?")).lower()
                amount = action.get("filled_amount") or action.get("amount", 0)
                price = action.get("price") or action.get("limit_price", 0)
                notional = action.get("notional_usd", 0)
                order_id = str(action.get("order_id", "?"))[:20]
                status = str(action.get("status", "?")).upper()[:15]
                intent = str(action.get("intent", "?")).lower()[:15]
                ts = str(action.get("ts", ""))[:19]

                try:
                    amount_f = float(amount)
                    amount_str = f"{amount_f:.4f}" if amount_f < 1 else f"{amount_f:.2f}"
                except:
                    amount_str = str(amount)[:12]

                try:
                    price_f = float(price)
                    price_str = f"{price_f:,.2f}" if price_f > 1000 else f"{price_f:.2f}" if price_f > 1 else f"{price_f:.4f}"
                except:
                    price_str = str(price)[:12]

                try:
                    notional_str = f"${float(notional):,.2f}"
                except:
                    notional_str = str(notional)[:12]

                if direction == "buy":
                    dir_emoji = "■"
                    dir_text = "■■■■" if intent in ("open", "increase") else "■■■■ ■■■■"
                    dir_en = "LONG" if intent in ("open", "increase") else "CLOSE SHORT"
                else:
                    dir_emoji = "■"
                    dir_text = "■■■■" if intent in ("open", "increase") else "■■■■ ■■■■"
                    dir_en = "SHORT" if intent in ("open", "increase") else "CLOSE LONG"

                status_emoji = "■" if status in ("FILLED", "CLOSED", "PARTIAL") else "■" if status == "DRY_RUN" else "■"

                lines = []
                lines.append(f"{dir_emoji} {asset} {dir_en} | {dir_text}")
                lines.append(f"■■■■■■■■■■■■■■■■■■■■■■■■")
                lines.append(f"■ ■■■■: {price_str} | ■■■■■: {amount_str}")
                lines.append(f"■ ■■■■: {notional_str}")
                lines.append(f"■ ■■■■■: {status_emoji} {status}")

                sl = action.get("sl") or action.get("stop_loss")
```

... ■■■■■ app/reporting/trade_reporter.py ...

```
                tp1 = action.get("tp1")
                tp2 = action.get("tp2")
                if sl or tp1:
                    lines.append(f"■■ ■■ ■■■/■■■:")
                    if sl:
                        try:
                            lines.append(f"   SL: {float(sl):,.2f}")
                        except:
                            lines.append(f"   SL: {sl}")
                    if tp1 and tp2:
                        try:
                            lines.append(f"   TP1: {float(tp1):,.2f} | TP2: {float(tp2):,.2f}")
                        except:
                            lines.append(f"   TP1: {tp1} | TP2: {tp2}")

                lines.append(f"■ ■■■■■: {order_id}")
                lines.append(f"■ ■■■: {intent}")
                if ts:
                    try:
                        dt = datetime.fromisoformat(ts.replace('Z', '+00:00'))
                        lines.append(f"■ ■■■■: {dt.strftime('%Y-%m-%d %H:%M:%S UTC')}")
                    except:
                        lines.append(f"■ ■■■■: {ts}")
                lines.append(f"■ ■■■■: {version}")
                lines.append(f"■■■■■■■■■■■■■■■■■■■■■■■■")
                lines.append(f"■■■■■■: AriaX Testnet | {inst}")
                return "\n".join(lines)
            except Exception as e:
                return f"■ ■■■■■■ {action.get('asset','?')} {action.get('direction','?')} {action.get('amount',0)} @ {action.get('pri

        def format_position(pos: dict) -> str:
            try:
```

```
        symbol = pos.get("symbol") or pos.get("instrument_name") or "?"
        size = float(pos.get("size") or 0)
        entry = float(pos.get("entry") or 0)
        mark = float(pos.get("mark") or 0)
        upnl = float(pos.get("upnl") or 0)
        lev = pos.get("lev") or 1
        side = "LONG" if size > 0 else "SHORT"
        emoji = "■" if size > 0 else "■"
        pnl_emoji = "■" if upnl >= 0 else "■"
        return f"{emoji} {symbol} {side} x{lev}\n■■■■■: {abs(size):.4f} | ■■■■: {entry:,.2f} | ■■■■: {mark:,.2f}\n{pnl_emoji
    except Exception as e:
        return f"Position error: {e}"

def format_balance(wallet: dict) -> str:
    try:
        balances = wallet.get("balances", {})
        usdt = balances.get("USDT", 0)
        equity = wallet.get("equity", usdt)
        lines = [f"■ ■■■■■■■", f"■■■■■■■■■■■■■■■■■■■■■■", f"USDT: {float(usdt):,.2f}", f"■■■■ ■■: {float(equity):,.2f} US
        for asset in ["BTC", "ETH"]:
            bal = balances.get(asset, 0)
            if abs(float(bal or 0)) > 0.0001:
                lines.append(f"{asset}: {float(bal):.6f}")
        return "\n".join(lines)
    except Exception as e:
        return f"Balance error: {e}"
```

```
"""
Telegram Module - Keyboards
Independent: Only keyboard layouts
"""
from typing import List

def main_keyboard(positions: List[dict] = None, dashboard_url: str = None) -> dict:
    keyboard = [
        [{"text": "■ ■■■■■", "callback_data": "panel:status"}, {"text": "■ ■■■■■■", "callback_data": "panel:positions"}, {"t
        [{"text": "■ ■■■■■■■■", "callback_data": "panel:sleeves"}, {"text": "■ ■■■■", "callback_data": "panel:stats"}, {"te
        [{"text": "■ Tick", "callback_data": "panel:tick"}, {"text": "■ ■■■■■ ■■■■", "callback_data": "panel:file"}, {"text"
        [{"text": "■ ■■■ ■■■■■■■", "callback_data": "panel:testtrade"}, {"text": "■ ■■■■■ ■■■", "callback_data": "panel:te
    ]
    if positions:
        active_pos = []
        for p in positions:
            try:
                sz = float(p.get("size_currency") or p.get("size") or 0)
                if abs(sz) > 1e-12:
                    active_pos.append(p)
            except:
                active_pos.append(p)
        if active_pos:
            close_row = []
            for p in active_pos[:6]:
                inst = p.get("instrument_name", "")
                asset = inst.replace("USDT", "").replace("USD", "") if inst else "UNKNOWN"
                direction = p.get("direction", "")
                icon = "■" if direction in ("buy", "long") else "■"
                close_row.append({"text": f"{icon} ■■■■ {asset}", "callback_data": f"close:{asset}"})
                if len(close_row) == 2:
                    keyboard.append(close_row)
                    close_row = []
            if close_row:
                keyboard.append(close_row)
            keyboard.append([{"text": "■ ■■■■ ■■■ ■■■■■■■■■", "callback_data": "panel:closeall"}])
    keyboard.append([{"text": "■■ Start", "callback_data": "panel:start"}, {"text": "■ Stop", "callback_data": "panel:stop"},
    if dashboard_url:
        keyboard.append([{"text": "■ ■■■■■■■ ■■", "url": dashboard_url}])
    return {"inline_keyboard": keyboard}
```

```
    """
    Telegram Module - Commands
    Independent: Only command parsing
    """
    def parse_command(text: str):
        """Parse /command and argument"""
        if not text or not text.startswith("/"):
            return None, ""
        parts = text.strip().split(maxsplit=1)
        cmd = parts[0].lower()
        arg = parts[1] if len(parts) > 1 else ""
        return cmd, arg
```

```
"""
Backward compatibility wrapper - imports from modular core
This file remains for compatibility, actual code in app/core/config.py
Each module independent - updating one doesn't affect others
"""
from app.core.config import Settings, get_settings, ALLOWED_ASSETS_FULL, ALLOWED_ASSETS_SMALL_CAP
__all__ = ["Settings", "get_settings", "ALLOWED_ASSETS_FULL", "ALLOWED_ASSETS_SMALL_CAP"]
```

■■■■: app/engine.py - 31903 ■■■■■■ - 677 ■■

```
"""
app/engine.py - Core Async Trading Engine v019 STABLE PRODUCTION
Integrated Fixes 1,2,3,4,5 + FIXED56 telegram + state management
Architecture: Asynchronous High-Availability Core (AriaX / Bybit V5 Compatible)
Fixes:
- Fix1 Maker/Post-Only Best Bid/Ask
- Fix2 Candle Close Gating $15 threshold
- Fix3 Sizing Guard BTC/ETH if <1000
- Fix4 Smart TP/SL No Zero-Distance 1.5 ATR
- Fix5 Full Async Engine 800ms
"""
from __future__ import annotations
import asyncio
import logging
import time
import json
import math
from datetime import datetime, timezone
from pathlib import Path
from typing import Dict, List
import pandas as pd

from app.config import Settings, get_settings
from app.ariax import AriaXAsyncClient, AriaXClient
from app.execution import maker_limit_price, plan_rebalance
from app.smart_tp_sl import calculate_initial_levels, evaluate_smart_exit

log = logging.getLogger("engine_v019")

def utcnow():
    return datetime.now(timezone.utc).isoformat()

class BotState:
    def __init__(self, path: str = "state/bot_state.json"):
        self.path = Path(path)
        self.path.parent.mkdir(parents=True, exist_ok=True)
        self.last_fill_at = None
        self.last_trade_at = {}
        self.position_sl_tp = {}
        self.events = []
        self.orders_log = []
        self.last_loop_at = None
        self.loop_count = 0
        self.load()

    def load(self):
        try:
            if self.path.exists():
                data = json.loads(self.path.read_text())
                self.position_sl_tp = data.get("position_sl_tp", {})
                self.last_trade_at = data.get("last_trade_at", {})
        except Exception as e:
            log.warning(f"State load failed: {e}")

    def save(self):
        try:
            data = {
                "position_sl_tp": self.position_sl_tp,
                "last_trade_at": self.last_trade_at,
                "last_loop_at": utcnow(),
```

... ■■■■■ app/engine.py ...

```
                "loop_count": self.loop_count
            }
            self.path.write_text(json.dumps(data, indent=2))
        except Exception as e:
            log.warning(f"State save failed: {e}")

    def push(self, level: str, msg: str):
        entry = {"ts": utcnow(), "level": level, "msg": msg}
        self.events.append(entry)
        self.events = self.events[-100:]
        log.info(f"[{level}] {msg}")


class SimpleAriaXTestTrader:
    def __init__(self, engine):
        self.engine = engine
        self._state = {"active": False, "status": "idle", "asset": "BTC", "seconds_remaining": None}
        self._timer = None
        self._lock = __import__("threading").Lock()
    def snapshot(self):
        import time
        s = self._state.copy()
        if s.get("active") and s.get("close_due"):
            remaining = max(0, s["close_due"] - time.time())
            s["seconds_remaining"] = remaining
        else:
            s["seconds_remaining"] = None
        s["hold_seconds"] = 60
        return s
    def start(self):
        with self._lock:
            if self._state.get("active"):
```

```
                    return self.snapshot()
                try:
                    import time, uuid, datetime
                    asset = "BTC"
                    instrument = "BTCUSDT"
                    self._state = {
                        "active": True, "status": "holding_dry", "asset": asset, "instrument": instrument,
                        "id": uuid.uuid4().hex[:8], "opened_at": datetime.datetime.now(datetime.timezone.utc).isoformat(),
                        "close_due": time.time() + 60, "amount": 0.001, "price": 80000.0, "reason": "dry_run simulated"
                    }
                    self._schedule_close()
                    self.engine.state.push("info", f"■ Test trade started: {asset}")
                    return self.snapshot()
                except Exception as e:
                    self._state = {"active": False, "status": f"failed: {e}", "asset": "BTC"}
                    return self.snapshot()
        def _schedule_close(self):
            import threading
            if self._timer:
                try: self._timer.cancel()
                except: pass
            def auto_close():
                try: self.request_close()
                except: pass
            self._timer = threading.Timer(60, auto_close)
            self._timer.daemon = True
            self._timer.start()
        def request_close(self):
```

... ■■■■■ app/engine.py ...

```
            with self._lock:
                if not self._state.get("active"):
                    return self.snapshot()
                try:
                    import datetime
                    asset = self._state.get("asset", "BTC")
                    instrument = self._state.get("instrument", "BTCUSDT")
                    self._state = {
                        "active": False, "status": "closed", "asset": asset, "instrument": instrument,
                        "closed_at": datetime.datetime.now(datetime.timezone.utc).isoformat(), "reason": "closed"
                    }
                    if self._timer:
                        try: self._timer.cancel()
                        except: pass
                    self.engine.state.push("info", f"■ Test trade closed: {asset}")
                    return self.snapshot()
                except Exception as e:
                    self._state["active"] = False
                    self._state["status"] = f"close_failed: {e}"
                    return self.snapshot()


    class TradingEngine:
        """
        v019 STABLE - Async Trading Engine with sync wrapper for compatibility
        """
        def __init__(self, settings: Settings = None):
            self.settings = settings or get_settings()
            self.state = BotState(self.settings.state_path)
            self.positions_state = {}  # For smart TP/SL v019
            self.last_candle_checked: Dict[str, str] = {}
            self._telegram_reporter = None
            self._reported_order_ids = set()
            self.pending_telegram_reports = []
            self._async_client: AriaXAsyncClient | None = None
            self._sync_client: AriaXClient | None = None
            self._last_async_market = {}
            self._last_async_positions = []
            self.candle_cache = {}
            try:
                self.test_trader = SimpleAriaXTestTrader(self)
            except:
                self.test_trader = None

        def set_telegram_reporter(self, reporter):
            self._telegram_reporter = reporter

        def _get_sync_client(self):
            if self._sync_client is None:
                self._sync_client = AriaXClient(
                    self.settings.ariax_base_url,
                    self.settings.ariax_api_key,
                    self.settings.ariax_api_secret,
                    self.settings.ariax_fallback_url
                )
            return self._sync_client

        def _get_async_client(self):
            if self._async_client is None:
                self._async_client = AriaXAsyncClient(
```

... ■■■■■ app/engine.py ...

```
                    self.settings.ariax_base_url,
                    self.settings.ariax_api_key,
                    self.settings.ariax_api_secret,
                    self.settings.ariax_fallback_url
                )
```

```
            return self._async_client

        def _log_fill(self, action: dict):
            oid = action.get("order_id")
            if oid and oid in self._reported_order_ids:
                return
            if oid:
                self._reported_order_ids.add(oid)
                if len(self._reported_order_ids) > 100:
                    self._reported_order_ids = set(list(self._reported_order_ids)[-100:])
            self.state.last_fill_at = utcnow()
            self.state.orders_log.append(action)
            self.state.orders_log = self.state.orders_log[-100:]
            try:
                if "ts" not in action:
                    action["ts"] = utcnow()
                if "filled_amount" not in action and "amount" in action:
                    action["filled_amount"] = action["amount"]
                self.pending_telegram_reports.append(action)
                self.pending_telegram_reports = self.pending_telegram_reports[-20:]
                self.state.push("info", f"■ Trade queued: {action.get('asset')} {action.get('direction')} {action.get('amount')} @
            except Exception as e:
                import logging
                logging.getLogger("engine").warning(f"Queue failed: {e}")
            try:
                asset = action.get("asset")
                if asset:
                    self.state.last_trade_at[asset] = time.time()
            except Exception:
                pass

        def flush_telegram_reports(self):
            if not self.pending_telegram_reports:
                return 0
            sent = 0
            try:
                reporter = getattr(self, '_telegram_reporter', None)
                if not reporter:
                    try:
                        from app.telegram_bot import get_reporter
                        reporter = get_reporter()
                    except Exception:
                        reporter = None
                if not reporter or not getattr(reporter, 'enabled', False):
                    self.pending_telegram_reports = []
                    return 0
                reports = self.pending_telegram_reports[:]
                self.pending_telegram_reports = []
                for action in reports:
                    try:
                        ok = reporter.send_trade_report(action)
                        if ok:
                            self.state.push("info", f"■ Trade report sent: {action.get('asset')} {action.get('direction')} {action.
                            sent += 1
                        else:
```

... ■■■■■ app/engine.py ...

```
                            self.state.push("error", f"■ Trade report failed: {action.get('asset')} err={getattr(reporter, 'last_er
                    except Exception as e:
                        self.state.push("error", f"flush failed {action.get('asset')}: {e}")
                return sent
            except Exception as e:
                import logging
                logging.getLogger("engine").warning(f"Flush failed: {e}")
                return sent

        def _calculate_atr_levels(self, df: pd.DataFrame, entry_price: float, side: int):
            """Backward compat wrapper for old calls"""
            try:
                from app.smart_tp_sl import _calculate_atr_levels_compat
                return _calculate_atr_levels_compat(df, entry_price, side)
            except:
                try:
                    high = df["high"]
                    low = df["low"]
                    close = df["close"]
                    tr = pd.concat([high - low, (high - close.shift()).abs(), (low - close.shift()).abs()], axis=1).max(axis=1)
                    atr = tr.rolling(14).mean().iloc[-1]
                    if pd.isna(atr):
                        atr = entry_price * 0.02
                    cfg = {"smart_sl_atr": 2.0, "smart_tp1_atr": 2.5, "smart_tp2_atr": 4.0}
                    levels = calculate_initial_levels(entry_price, side, atr, cfg)
                    return {"sl": levels["sl"], "tp1": levels["tp1"], "tp2": levels["tp2"], "atr": atr}
                except:
                    atr = entry_price * 0.02
                    return {"sl": entry_price * (0.98 if side>0 else 1.02), "tp1": entry_price * (1.05 if side>0 else 0.95), "tp2":

        # === v019 ASYNC CORE (Fix 5) ===
        async def run_cycle_async(self):
            """v019 Fix 5: Concurrent parallel execution for all instruments - 800ms target"""
            start_ts = time.time()
            try:
                settings = self.settings
                client = self._get_async_client()

                # Fix 3: Dynamic assets
                active_assets = settings.effective_assets if hasattr(settings, 'effective_assets') else [a.strip() for a in setting
```

```
                # FIX: Cancel all open orders if >10 to prevent 69 orders accumulation
                try:
                    import httpx
                    base = settings.ariax_base_url
                    headers = {"X-API-Key": settings.ariax_api_key, "X-API-Secret": settings.ariax_api_secret}
                    try:
                        r = httpx.get(f"{base}/api/orders", headers=headers, timeout=5)
                        orders = r.json().get("data", []) if r.status_code == 200 else []
                        if len(orders) > 10:
                            import logging
                            logging.getLogger("engine").info(f"Found {len(orders)} open orders, canceling all")
                            for o in orders:
                                try:
                                    httpx.post(f"{base}/api/cancel", headers=headers, json={"id": o.get("id")}, timeout=5)
                                except:
                                    pass
                            self.state.push("info", f"■ Canceled {len(orders)} old orders")
                    except Exception as e:
                        import logging
```

... ■■■■■ app/engine.py ...

```
                        logging.getLogger("engine").warning(f"Cancel all failed: {e}")
                except Exception as e:
                    import logging
                    logging.getLogger("engine").warning(f"Order cleanup failed: {e}")

                # 1. Concurrently fetch account and positions
                account_task = client.account_summary("USDT")
                positions_task = client.positions("USDT")
                account, positions = await asyncio.gather(account_task, positions_task)

                # 2. Concurrently fetch orderbook and candles for all active assets
                tasks = []
                for asset in active_assets:
                    symbol = settings.instrument_for(asset)
                    order_sym = settings.order_symbol_for(asset) if hasattr(settings, 'order_symbol_for') else symbol
                    tasks.append(client.order_book(symbol))
                    tasks.append(client.candles(symbol, hours=100))

                results = await asyncio.gather(*tasks, return_exceptions=True)

                market_data = {}
                for i, asset in enumerate(active_assets):
                    try:
                        book_res = results[i*2]
                        candle_res = results[i*2+1]

                        if isinstance(book_res, Exception):
                            book = {"best_bid": 0, "best_ask": 0, "state": "unknown", "bids": [], "asks": []}
                        else:
                            book = book_res

                        if isinstance(candle_res, Exception) or not isinstance(candle_res, pd.DataFrame):
                            df = pd.DataFrame()
                        else:
                            df = candle_res

                        market_data[asset] = {"book": book, "candles": df}
                    except Exception as e:
                        log.warning(f"Market data parse failed for {asset}: {e}")
                        market_data[asset] = {"book": {"best_bid":0,"best_ask":0,"state":"unknown"}, "candles": pd.DataFrame()}

                pos_map = {p.get("instrument_name") or p.get("symbol"): p for p in positions}
                self._last_async_market = market_data
                self._last_async_positions = positions

                # 3. Process each asset with v019 logic
                for asset in active_assets:
                    symbol = settings.instrument_for(asset)
                    order_sym = settings.order_symbol_for(asset) if hasattr(settings, 'order_symbol_for') else symbol
                    book = market_data[asset]["book"]
                    df = market_data[asset]["candles"]

                    if df.empty or book.get("state") not in ("open", "unknown") and len(book.get("bids",[])) == 0:
                        # Allow unknown state for dryclean mock
                        if df.empty:
                            continue

                    current_pos = pos_map.get(symbol, {})
                    current_qty = float(current_pos.get("size", 0.0))
                    mark_price = book.get("best_bid", 0.0) or book.get("best_ask", 0.0) or 0.0
```

... ■■■■■ app/engine.py ...

```
                    if mark_price <= 0:
                        # Fallback to last close
                        try:
                            mark_price = float(df["close"].iloc[-1]) if not df.empty else 0.0
                        except:
                            mark_price = 0.0

                    if mark_price <= 0:
                        continue

                    # Fix 4: Check Active TP/SL & Trailing Management
                    if abs(current_qty) > 1e-8 and asset in self.positions_state:
                        atr = self.positions_state[asset].get("atr", mark_price * 0.02)
                        exit_eval = evaluate_smart_exit(mark_price, self.positions_state[asset], atr)
```

```
                    if exit_eval.get("trigger_exit"):
                        log.warning(f"Closing position on {asset} due to {exit_eval['trigger_exit']} @ {mark_price}")
                        close_side = "sell" if current_qty > 0 else "buy"
                        try:
                            price = maker_limit_price(book, close_side)
                            await client.place_order(order_sym, close_side, abs(current_qty), price, post_only=False, reduce_on
                            self._log_fill({
                                "asset": asset,
                                "instrument": symbol,
                                "direction": close_side,
                                "amount": abs(current_qty),
                                "price": price,
                                "notional_usd": abs(current_qty) * price,
                                "order_id": f"sl_{asset}_{int(time.time())}",
                                "status": "closed",
                                "intent": "close"
                            })
                        except Exception as e:
                            log.error(f"Failed to close {asset}: {e}")
                        self.positions_state.pop(asset, None)
                        self.state.position_sl_tp.pop(asset, None)
                        continue

                    if exit_eval.get("close_fraction", 0) > 0:
                        partial_qty = round(abs(current_qty) * exit_eval["close_fraction"], 4)
                        close_side = "sell" if current_qty > 0 else "buy"
                        try:
                            price = maker_limit_price(book, close_side)
                            await client.place_order(order_sym, close_side, partial_qty, price, post_only=settings.post_only, r
                            self._log_fill({
                                "asset": asset,
                                "instrument": symbol,
                                "direction": close_side,
                                "amount": partial_qty,
                                "price": price,
                                "notional_usd": partial_qty * price,
                                "order_id": f"tp_{asset}_{int(time.time())}",
                                "status": "partial",
                                "intent": "close"
                            })
                        except Exception as e:
                            log.error(f"Failed partial close {asset}: {e}")

                    # Update SL in state
                    if exit_eval.get("new_sl") and exit_eval["new_sl"] != self.positions_state[asset].get("sl"):
```

... ■■■■■ app/engine.py ...

```
                        self.positions_state[asset]["sl"] = exit_eval["new_sl"]
                        self.state.position_sl_tp[asset] = self.positions_state[asset]

                # Fix 2: Candle Close Gating - only trade on hourly close
                if settings.trade_on_candle_close_only and not df.empty:
                    try:
                        last_dt = str(df["dt"].iloc[-1])
                        if self.last_candle_checked.get(asset) == last_dt:
                            continue  # Already analyzed this candle
                        self.last_candle_checked[asset] = last_dt
                    except:
                        pass

                # Quantitative Signal: EMA 20/50
                if df.empty or len(df) < 50:
                    continue

                try:
                    close = df["close"]
                    ema20 = close.ewm(span=20).mean().iloc[-1]
                    ema50 = close.ewm(span=50).mean().iloc[-1]
                except:
                    continue

                desired_direction = 1 if ema20 > ema50 else (-1 if ema20 < ema50 else 0)

                if desired_direction == 0:
                    continue

                # Sizing - Fix 3
                allocated_usd = (settings.capital_usd / len(active_assets)) * settings.lev_cap
                desired_qty = (allocated_usd / mark_price) * desired_direction

                # Execute Maker Rebalance - Fix 1 & 2
                plan = plan_rebalance(
                    current=current_qty,
                    desired=desired_qty,
                    price=mark_price,
                    contract_step=0.001,
                    min_trade_amount=0.001,
                    rebalance_threshold_usd=settings.rebalance_notional_usd
                )

                if plan.status == "planned" and plan.amount > 0:
                    try:
                        limit_px = maker_limit_price(book, plan.direction)
                        log.info(f"Submitting Maker Order for {asset}: {plan.direction} {plan.amount} @ {limit_px} (threshold $

                        if not settings.dry_run and settings.trading_enabled:
                            await client.place_order(
```

```
                                symbol=order_sym,
                                side=plan.direction,
                                qty=plan.amount,
                                price=limit_px,
                                post_only=settings.post_only,
                                reduce_only=plan.reduce_only
                            )

                            self._log_fill({
                                "asset": asset,
```

... ■■■■■ app/engine.py ...

```
                                "instrument": symbol,
                                "direction": plan.direction,
                                "amount": plan.amount,
                                "filled_amount": plan.amount,
                                "price": limit_px,
                                "notional_usd": plan.amount * limit_px,
                                "order_id": f"{asset}_{int(time.time())}",
                                "status": "filled" if not settings.dry_run else "dry_run",
                                "intent": "open" if not plan.reduce_only else "close"
                            })

                            # Register for Smart TP/SL - Fix 4
                            if not plan.reduce_only and settings.smart_tp_sl_enabled:
                                atr = mark_price * 0.02
                                try:
                                    # Calculate ATR from df if possible
                                    high = df["high"]
                                    low = df["low"]
                                    close_s = df["close"]
                                    tr = pd.concat([high - low, (high - close_s.shift()).abs(), (low - close_s.shift()).abs()], axi
                                    atr_calc = tr.rolling(14).mean().iloc[-1]
                                    if not pd.isna(atr_calc) and atr_calc > 0:
                                        atr = float(atr_calc)
                                except:
                                    pass

                                self.positions_state[asset] = calculate_initial_levels(
                                    entry=limit_px,
                                    side=1 if plan.direction == "buy" else -1,
                                    atr=atr,
                                    cfg=settings.model_dump() if hasattr(settings, 'model_dump') else settings.__dict__
                                )
                                self.state.position_sl_tp[asset] = self.positions_state[asset]
                                self.state.push("info", f"SMART TP/SL registered for {asset}: SL={self.positions_state[asset]['sl']
                        except Exception as ex:
                            log.error(f"Failed to submit order for {asset}: {ex}")
                            self.state.push("error", f"Order failed {asset}: {ex}")

                elapsed = time.time() - start_ts
                log.info(f"v019 cycle completed in {elapsed:.3f}s for {len(active_assets)} assets (target <0.8s)")
                self.state.loop_count += 1
                self.state.last_loop_at = utcnow()
                self.state.save()
                return True

            except Exception as e:
                log.exception(f"Unhandled async cycle error: {e}")
                self.state.push("error", f"Cycle error: {e}")
                return False

        # Sync wrapper for backward compatibility with main.py
        def run_cycle(self):
            """Sync wrapper that runs async cycle"""
            try:
                loop = asyncio.new_event_loop()
                asyncio.set_event_loop(loop)
                result = loop.run_until_complete(self.run_cycle_async())
                loop.close()
                return result
            except Exception as e:
```

... ■■■■■ app/engine.py ...

```
                log.exception(f"Sync wrapper error: {e}")
                # Fallback to old sync logic if async fails
                try:
                    return self._run_cycle_sync_fallback()
                except Exception as e2:
                    log.exception(f"Fallback also failed: {e2}")
                    return False

        def _run_cycle_sync_fallback(self):
            """Minimal sync fallback to keep bot alive if async fails"""
            try:
                client = self._get_sync_client()
                settings = self.settings
                active_assets = settings.effective_assets if hasattr(settings, 'effective_assets') else [a.strip() for a in setting

                account = client.account_summary("USDT")
                positions = client.positions("USDT")
                pos_map = {p.get("instrument_name"): p for p in positions}

                for asset in active_assets[:2]:  # Only 2 for fallback speed
                    symbol = settings.instrument_for(asset)
                    order_sym = settings.order_symbol_for(asset) if hasattr(settings, 'order_symbol_for') else symbol
                    try:
```

```
                    book = client.order_book(symbol)
                    df = client.candles(symbol, hours=100)
                    if df.empty:
                        continue

                    current_pos = pos_map.get(symbol, {})
                    current_qty = float(current_pos.get("size", 0))
                    mark_price = book.get("best_bid", 0) or 0

                    if mark_price <= 0:
                        continue

                    # Simple EMA signal
                    close = df["close"]
                    if len(close) < 50:
                        continue
                    ema20 = close.ewm(span=20).mean().iloc[-1]
                    ema50 = close.ewm(span=50).mean().iloc[-1]
                    direction = 1 if ema20 > ema50 else -1

                    allocated = (settings.capital_usd / len(active_assets)) * settings.lev_cap
                    desired = (allocated / mark_price) * direction

                    plan = plan_rebalance(current_qty, desired, mark_price, 0.001, 0.001, settings.rebalance_notional_usd)
                    if plan.status == "planned" and plan.amount > 0:
                        limit_px = maker_limit_price(book, plan.direction)
                        if not settings.dry_run:
                            client.place_order(order_sym, plan.direction, plan.amount, limit_px, post_only=True, reduce_only=pl
                        self._log_fill({
                            "asset": asset,
                            "instrument": symbol,
                            "direction": plan.direction,
                            "amount": plan.amount,
                            "price": limit_px,
                            "notional_usd": plan.amount * limit_px,
                            "order_id": f"{asset}_{int(time.time())}",
                            "status": "filled",
```

... ■■■■■ app/engine.py ...

```
                            "intent": "open" if not plan.reduce_only else "close"
                        })
                    except Exception as e:
                        log.warning(f"Fallback asset {asset} failed: {e}")
                        continue

                self.state.loop_count += 1
                self.state.save()
                return True
            except Exception as e:
                log.exception(f"Fallback cycle error: {e}")
                return False

        def close_position(self, asset: str):
            """Close position - REAL TRADING with exchange order"""
            try:
                self.state.push("info", f"■ Close position requested for {asset}")
                # Try to close on exchange if real trading
                if not self.settings.effective_dry_run and self.settings.trading_enabled:
                    try:
                        import httpx
                        base = self.settings.ariax_base_url
                        headers = {"X-API-Key": self.settings.ariax_api_key, "X-API-Secret": self.settings.ariax_api_secret}
                        # Get positions to find size
                        r = httpx.get(f"{base}/api/positions", headers=headers, timeout=10)
                        data = r.json()
                        positions = data.get("data", []) if isinstance(data, dict) else []
                        target_pos = None
                        for pos in positions:
                            sym = pos.get("symbol", "")
                            if asset.upper() in sym.upper():
                                target_pos = pos
                                break
                        if target_pos:
                            size = float(target_pos.get("size", 0))
                            if abs(size) > 1e-8:
                                side = "sell" if size > 0 else "buy"
                                symbol = target_pos.get("symbol")
                                body = {"symbol": symbol, "side": side, "qty": abs(size), "type": "market"}
                                r2 = httpx.post(f"{base}/api/order", headers=headers, json=body, timeout=10)
                                self.state.push("info", f"■ Close order {asset} {side} {abs(size)} resp={r2.text[:200]}")
                            else:
                                self.state.push("info", f"No position size for {asset}")
                        else:
                            self.state.push("info", f"No position found for {asset} on exchange")
                    except Exception as e:
                        self.state.push("error", f"Close exchange failed {asset}: {e}")

                # Clean local state
                if asset in self.positions_state:
                    self.positions_state.pop(asset)
                if asset in self.state.position_sl_tp:
                    self.state.position_sl_tp.pop(asset)

                self.state.push("info", f"■ Position {asset} closed (local state cleared)")
                return True
            except Exception as e:
                self.state.push("error", f"Close failed {asset}: {e}")
```

```
                return False
```

... ■■■■■ app/engine.py ...

```
        def auto_register_smart(self, asset: str, entry: float, side: int, atr: float):
            """v018 compat - Auto-register SMART TP/SL"""
            cfg = self.settings.model_dump() if hasattr(self.settings, 'model_dump') else self.settings.__dict__
            self.positions_state[asset] = calculate_initial_levels(entry, side, atr, cfg)
            self.state.position_sl_tp[asset] = self.positions_state[asset]
            self.state.push("info", f"Auto-register SMART TP/SL for {asset}: Entry={entry} SL={self.positions_state[asset]['sl']}")
            return self.positions_state[asset]


    # Global engine instance for main.py compatibility
    _engine_instance = None

    def get_engine(settings=None):
        global _engine_instance
        if _engine_instance is None:
            _engine_instance = TradingEngine(settings)
        return _engine_instance
```

```
"""
AriaX client - FIXED31 FINAL - robust wallet and positions parsing
"""
from __future__ import annotations
import time
import logging
from typing import List, Optional, Dict
import httpx
import pandas as pd

log = logging.getLogger("ariax")

class AriaXAPIError(Exception):
    def __init__(self, path: str, code, msg: str):
        self.path = path
        self.code = code
        super().__init__(f"{path} {code}: {msg}")

class AriaXClient:
    def __init__(self, base_url: str, api_key: str, api_secret: str, fallback_url: Optional[str] = None):
        self.base_url = (base_url or "").rstrip("/")
        self.fallback_url = (fallback_url or self.base_url).rstrip("/")
        self.api_key = api_key
        self.api_secret = api_secret
        self._http = httpx.Client(timeout=15.0)
        self._instrument_cache: Dict[str, tuple] = {}

    def _request(self, method: str, path: str, params: Optional[dict] = None, json_body: Optional[dict] = None, use_v5: bool =
        urls = [self.base_url, self.fallback_url] if self.fallback_url != self.base_url else [self.base_url]
        last_exc = None
        for base in urls:
            url = f"{base}{path}"
            try:
                if is_public:
                    if method == "GET":
                        resp = self._http.get(url, params=params)
                    else:
                        resp = self._http.request(method, url, params=params, json=json_body)
                else:
                    headers = {"X-API-Key": self.api_key, "X-API-Secret": self.api_secret}
                    if method == "GET":
                        resp = self._http.get(url, params=params, headers=headers)
                    else:
                        resp = self._http.request(method, url, params=params, json=json_body, headers=headers)

                if resp.status_code == 404 and "no-server" in resp.headers.get("x-render-routing", ""):
                    last_exc = AriaXAPIError(path, 404, "no-server")
                    continue

                try:
                    data = resp.json()
                except:
                    if resp.status_code >= 400:
                        raise AriaXAPIError(path, resp.status_code, f"HTTP {resp.status_code}")
                    return {}

                if isinstance(data, dict) and "retCode" in data:
                    if data.get("retCode") != 0:
                        raise AriaXAPIError(path, data.get("retCode"), data.get("retMsg", "error"))
                    return data
```

... ■■■■■ app/ariax.py ...

```
                if isinstance(data, dict) and "ok" in data:
                    if not data.get("ok"):
                        raise AriaXAPIError(path, data.get("error") or -1, str(data))
                    return data

                return data

            except AriaXAPIError as exc:
                if exc.code in (404, "transport") or "no-server" in str(exc):
                    last_exc = exc
                    continue
                raise
            except httpx.HTTPError as exc:
                last_exc = AriaXAPIError(path, "transport", type(exc).__name__)
                continue

        if last_exc:
            raise last_exc
        raise AriaXAPIError(path, -1, "all endpoints failed")

    def ticker(self, instrument: str) -> dict:
        try:
            data = self._request("GET", "/v5/market/tickers", params={"category": "linear", "symbol": instrument}, is_public=Tr
            lst = data.get("result", {}).get("list", [])
            return lst[0] if lst else {}
        except:
            return {}

    def order_book(self, instrument: str, depth: int = 5) -> dict:
        try:
            limit = 25
            data = self._request("GET", "/v5/market/orderbook", params={"category": "linear", "symbol": instrument, "limit": li
```

```
            res = data.get("result", {})
            bids = [[float(p), float(q)] for p, q in res.get("b", [])]
            asks = [[float(p), float(q)] for p, q in res.get("a", [])]
            ticker = self.ticker(instrument)
            mark = float(ticker.get("markPrice") or ticker.get("lastPrice") or 0)
            return {
                "bids": bids, "asks": asks,
                "timestamp": res.get("ts", int(time.time()*1000)),
                "mark_price": mark,
                "state": "open" if bids or asks else "unknown",
            }
        except Exception as exc:
            return {"bids": [], "asks": [], "timestamp": int(time.time()*1000), "mark_price": 0, "state": "unknown"}

    def instrument(self, instrument: str) -> dict:
        import time as tm
        now = tm.monotonic()
        cached = self._instrument_cache.get(instrument)
        if cached and now - cached[0] < 300:
            return dict(cached[1])

        try:
            data = self._request("GET", "/v5/market/instruments-info", params={"category": "linear", "symbol": instrument}, is_
            lst = data.get("result", {}).get("list", [])
            if not lst:
                result = {"min_trade_amount": 0.001, "contract_size": 0.001, "tick_size": 0.01, "min_notional": 5.0}
            else:
```

... ■■■■■ app/ariax.py ...

```
                info = lst[0]
                lot = info.get("lotSizeFilter", {})
                price_filter = info.get("priceFilter", {})
                result = {
                    "min_trade_amount": float(lot.get("minOrderQty") or 0.001),
                    "contract_size": float(lot.get("qtyStep") or 0.001),
                    "tick_size": float(price_filter.get("tickSize") or 0.01),
                    "min_notional": float(lot.get("minNotionalValue") or 5.0),
                }
        except:
            result = {"min_trade_amount": 0.001, "contract_size": 0.001, "tick_size": 0.01, "min_notional": 5.0}

        self._instrument_cache[instrument] = (now, result)
        return dict(result)

    def candles(self, instrument: str, hours: int = 1000, resolution: str = "60") -> pd.DataFrame:
        try:
            import time as tm
            all_rows = []
            end_ms = int(tm.time() * 1000)
            remaining = max(48, hours)

            for _ in range(3):
                if remaining <= 0:
                    break
                limit = min(1000, remaining)
                try:
                    data = self._request("GET", "/v5/market/kline", params={
                        "category": "linear", "symbol": instrument, "interval": "60", "limit": limit, "end": end_ms
                    }, is_public=True)
                    rows = data.get("result", {}).get("list", [])
                    if not rows:
                        break
                    all_rows.extend(rows)
                    oldest_ts = min(int(r[0]) for r in rows)
                    end_ms = oldest_ts - 1
                    remaining -= len(rows)
                    if len(rows) < limit:
                        break
                    tm.sleep(0.1)
                except:
                    break

            if not all_rows:
                return pd.DataFrame()

            df = pd.DataFrame(all_rows, columns=["timestamp", "open", "high", "low", "close", "volume", "turnover"])
            df["timestamp"] = pd.to_numeric(df["timestamp"], errors="coerce")
            for col in ["open", "high", "low", "close", "volume"]:
                df[col] = pd.to_numeric(df[col], errors="coerce")
            df["dt"] = pd.to_datetime(df["timestamp"], unit="ms", utc=True)
            df = df.sort_values("dt").reset_index(drop=True)
            return df[["dt", "open", "high", "low", "close", "volume"]]
        except Exception as exc:
            log.warning(f"candles failed for {instrument}: {exc}")
            return pd.DataFrame()

    def account_summary(self, currency: str = "USDT") -> dict:
        try:
            data = self._request("GET", "/api/wallet", use_v5=False)
```

... ■■■■■ app/ariax.py ...

```
        except Exception as e:
            log.warning(f"account_summary _request failed: {e}")
            return {currency: {"equity": 0, "available": 0, "balance": 0}}

        try:
            if not isinstance(data, dict):
```

```
                    return {currency: {"equity": 0, "available": 0, "balance": 0}}

                # Try equity directly
                equity = data.get("equity")
                if equity is not None:
                    try:
                        eq = float(equity)
                        if eq > 0:
                            free = float(data.get("free_margin") or 0)
                            fut_bal = 0
                            try:
                                fb = data.get("futures", {}).get("balances", {})
                                if isinstance(fb, dict):
                                    fut_bal = float(fb.get("USDT") or 0)
                            except:
                                pass
                            available = free if free > 0 else fut_bal if fut_bal > 0 else eq
                            balance = fut_bal if fut_bal > 0 else eq
                            return {currency: {"equity": eq, "available": available, "balance": balance}}
                    except:
                        pass

                # Fallback futures
                try:
                    fb = data.get("futures", {}).get("balances", {})
                    if isinstance(fb, dict):
                        fut = float(fb.get("USDT") or 0)
                        if fut > 0:
                            return {currency: {"equity": fut, "available": fut, "balance": fut}}
                except:
                    pass

                # Fallback spot
                try:
                    sb = data.get("balances", {})
                    if isinstance(sb, dict):
                        spot = float(sb.get("USDT") or 0)
                        if spot > 0:
                            return {currency: {"equity": spot, "available": spot, "balance": spot}}
                except:
                    pass

                return {currency: {"equity": 0, "available": 0, "balance": 0}}
            except Exception as e:
                log.warning(f"account_summary parse failed: {e}")
                return {currency: {"equity": 0, "available": 0, "balance": 0}}

        def positions(self, currency: str = "USDT") -> List[dict]:
            try:
                data = self._request("GET", "/api/positions", use_v5=False)
            except Exception as e:
                log.warning(f"positions _request failed: {e}")
                return []
```

... ■■■■■ app/ariax.py ...

```
                try:
                    raw_positions = []
                    if isinstance(data, dict) and "data" in data:
                        raw_positions = data.get("data", [])
                    elif isinstance(data, list):
                        raw_positions = data
                    else:
                        return []

                    normalized = []
                    for p in raw_positions:
                        try:
                            symbol = p.get("symbol") or p.get("instrument_name") or ""
                            asset = symbol.replace("USDT", "").replace("USD", "")
                            instrument = asset + "USDT" if symbol.endswith("USD") and not symbol.endswith("USDT") else symbol
                            if not instrument.endswith("USDT"):
                                instrument = asset + "USDT"

                            size = float(p.get("size") or 0)
                            entry = float(p.get("entry") or p.get("average_price") or 0)
                            mark = float(p.get("mark") or p.get("mark_price") or 0)
                            upnl = float(p.get("upnl") or p.get("total_profit_loss") or 0)
                            lev = p.get("lev") or p.get("leverage") or 5
                            direction = "sell" if size < 0 else "buy" if size > 0 else ""

                            normalized.append({
                                "instrument_name": instrument,
                                "symbol": symbol,
                                "size": size,
                                "size_currency": size,
                                "direction": direction,
                                "average_price": entry,
                                "mark_price": mark,
                                "total_profit_loss": upnl,
                                "floating_profit_loss": upnl,
                                "leverage": lev,
                                "entry": entry,
                                "mark": mark,
                                "upnl": upnl,
                                "margin": p.get("margin"),
                                "liq": p.get("liq"),
                                "sl": p.get("sl"),
```

```
                        "tp": p.get("tp"),
                    })
                except Exception as e:
                    continue

            return normalized
        except Exception as e:
            log.warning(f"positions parse failed: {e}")
            return []

    def open_orders(self, currency: str = "USDT") -> List[dict]:
        return []

    def orders_by_label(self, label: str) -> List[dict]:
        return []

    def cancel_order(self, order_id: str) -> dict:
        try:
```

... ■■■■■ app/ariax.py ...

```
            data = self._request("POST", "/api/cancel", json_body={"id": order_id}, use_v5=False)
            return data
        except:
            return {}


    def close_position(self, symbol: str) -> dict:
        """Close position - FIXED56: meet min qty and 5$ notional with reduce_only"""
        try:
            import math
            positions = self.positions("USDT")
            target_pos = None
            for p in positions:
                inst = p.get("instrument_name", "")
                if inst == symbol or inst.replace("USDT","") == symbol.replace("USDT","").replace("USD",""):
                    target_pos = p
                    break
                if symbol.replace("USDT","").upper() in inst.upper():
                    target_pos = p
                    break
            if not target_pos:
                for p in positions:
                    if p.get("symbol","").upper() == symbol.replace("USDT","USD").upper() or p.get("instrument_name","").upper(
                        target_pos = p
                        break
            if not target_pos:
                return {"ok": False, "error": f"No position found for {symbol}", "positions": len(positions)}

            size = float(target_pos.get("size") or target_pos.get("size_currency") or 0)
            if abs(size) < 1e-9:
                return {"ok": False, "error": f"Position size 0 for {symbol}"}

            # Get instrument meta for min qty and step
            try:
                meta = self.instrument(symbol)
                step = float(meta.get("contract_size") or meta.get("qty_step") or 0.001)
                minimum = float(meta.get("min_trade_amount") or meta.get("min_qty") or step)
            except:
                step = 0.001
                minimum = 0.001

            # Get current price for notional calc
            try:
                book = self.order_book(symbol, 1)
                price = float(book.get("mark_price") or book.get("bids",[[0]])[0][0] or 0)
            except:
                price = float(target_pos.get("mark_price") or target_pos.get("average_price") or 0)

            # Calculate required qty to meet 5.1$ notional
            min_qty_notional = 5.1 / price if price>0 else minimum
            if step>0:
                min_qty_notional = math.ceil(min_qty_notional / step) * step
            required_min = max(minimum, min_qty_notional, step)

            # For close, use max(current, required_min) with reduce_only=True
            # Exchange with reduce_only should close only up to current size
            qty = max(abs(size), required_min)
            # Ensure qty is multiple of step
            if step>0:
                qty = math.ceil(qty / step) * step
```

... ■■■■■ app/ariax.py ...

```
            side = "sell" if size > 0 else "buy"

            # Try v5 first (might have different min checks), fallback to /api/order
            try:
                # Try v5 order create
                body_v5 = {
                    "symbol": symbol.replace("USDT","USD") if symbol.endswith("USDT") else symbol,
                    "side": side,
                    "orderType": "Market",
                    "qty": str(qty),
                    "reduceOnly": True,
                }
                result = self._request("POST", "/v5/order/create", json_body=body_v5, use_v5=True)
                return {"ok": True, "symbol": symbol, "side": side, "qty": qty, "result": result, "closed_size": size, "via": "
            except Exception as e_v5:
```

```
                    # Fallback to /api/order
                    try:
                        result = self._place_order(symbol, side, qty, order_type="market", reduce_only=True, label=f"close_{symbol}
                        return {"ok": True, "symbol": symbol, "side": side, "qty": qty, "result": result, "closed_size": size, "via
                    except Exception as e_api:
                        return {"ok": False, "error": f"v5 failed: {e_v5}, api failed: {e_api}", "symbol": symbol, "qty": qty, "siz
            except Exception as e:
                import traceback
                return {"ok": False, "error": str(e), "tb": traceback.format_exc()[:500]}

        def _place_order(self, symbol: str, side: str, qty: float, order_type: str = "market", price: Optional[float] = None, reduc
            body = {
                "symbol": symbol.replace("USDT", "USD") if symbol.endswith("USDT") else symbol,
                "side": side,
                "qty": qty,
                "type": order_type,
                "reduce_only": reduce_only,
            }
            if price is not None:
                body["price"] = price
            if label:
                body["label"] = label

            try:
                data = self._request("POST", "/api/order", json_body=body, use_v5=False)
                return data
            except Exception as e:
                raise e


    # === v019 ASYNC CLIENT (Fix 5) ===
    import asyncio as _asyncio

    class AriaXAPIError(Exception):
        def __init__(self, path: str, code, msg: str):
            self.path = path
            self.code = code
            super().__init__(f"{path} [{code}]: {msg}")

    class AriaXAsyncClient:
        def __init__(self, base_url: str, api_key: str, api_secret: str, fallback_url: str = None):
            self.base_url = (base_url or "").rstrip("/")
            self.fallback_url = (fallback_url or self.base_url).rstrip("/")
            self.api_key = api_key
```

... ■■■■■ app/ariax.py ...

```
            self.api_secret = api_secret
            self._http = None

        def _get_http(self):
            if self._http is None:
                import httpx
                self._http = httpx.AsyncClient(timeout=10.0)
            return self._http

        async def close(self):
            if self._http:
                await self._http.aclose()
                self._http = None

        async def _request(self, method: str, path: str, params=None, json_body=None, is_public: bool = False) -> dict:
            import httpx
            http = self._get_http()
            urls = [self.base_url, self.fallback_url] if self.fallback_url != self.base_url else [self.base_url]
            last_exc = None

            headers = {}
            if not is_public:
                headers = {"X-API-Key": self.api_key, "X-API-Secret": self.api_secret}

            for base in urls:
                url = f"{base}{path}"
                try:
                    resp = await http.request(method, url, params=params, json=json_body, headers=headers)
                    if resp.status_code >= 400:
                        raise AriaXAPIError(path, resp.status_code, f"HTTP {resp.status_code}")
                    data = resp.json()
                    # Check Bybit-style retCode
                    if isinstance(data, dict) and data.get("retCode", 0) != 0:
                        raise AriaXAPIError(path, data.get("retCode"), data.get("retMsg", "Unknown Error"))
                    # Check legacy ok field - CRITICAL FIX for insufficient balance etc
                    if isinstance(data, dict) and "ok" in data and not data.get("ok"):
                        raise AriaXAPIError(path, data.get("error") or -1, str(data.get("error") or data))
                    return data
                except Exception as exc:
                    last_exc = exc
                    continue

            raise last_exc or AriaXAPIError(path, -1, "All endpoints failed")

        async def order_book(self, instrument: str, depth: int = 5) -> dict:
            try:
                import time
                data = await self._request("GET", "/v5/market/orderbook", params={"category": "linear", "symbol": instrument, "limi
                res = data.get("result", {})
                bids = [[float(p), float(q)] for p, q in res.get("b", [])]
                asks = [[float(p), float(q)] for p, q in res.get("a", [])]
```

```
            return {
                "bids": bids, "asks": asks,
                "timestamp": res.get("ts", int(time.time() * 1000)),
                "best_bid": bids[0][0] if bids else 0.0,
                "best_ask": asks[0][0] if asks else 0.0,
                "state": "open" if bids and asks else "closed"
            }
        except Exception as e:
            import logging
```

... ■■■■■ app/ariax.py ...

```
            logging.getLogger("ariax_async").error(f"Failed to fetch orderbook for {instrument}: {e}")
            return {"bids": [], "asks": [], "best_bid": 0.0, "best_ask": 0.0, "state": "unknown"}

    async def candles(self, instrument: str, hours: int = 150):
        try:
            import pandas as pd
            data = await self._request("GET", "/v5/market/kline", params={"category": "linear", "symbol": instrument, "interval
            rows = data.get("result", {}).get("list", [])
            if not rows:
                return pd.DataFrame()
            df = pd.DataFrame(rows, columns=["timestamp", "open", "high", "low", "close", "volume", "turnover"])
            for col in ["open", "high", "low", "close", "volume"]:
                df[col] = pd.to_numeric(df[col], errors="coerce")
            df["dt"] = pd.to_datetime(pd.to_numeric(df["timestamp"]), unit="ms", utc=True)
            return df.sort_values("dt").reset_index(drop=True)[["dt", "open", "high", "low", "close", "volume"]]
        except Exception as exc:
            import logging
            logging.getLogger("ariax_async").warning(f"Failed to fetch candles for {instrument}: {exc}")
            import pandas as pd
            return pd.DataFrame()

    async def account_summary(self, currency: str = "USDT") -> dict:
        try:
            data = await self._request("GET", "/api/wallet", is_public=False)
            equity = float(data.get("equity") or 0.0)
            available = float(data.get("free_margin") or data.get("available") or 0.0)
            return {currency: {"equity": equity, "available": available}}
        except Exception as exc:
            import logging
            logging.getLogger("ariax_async").warning(f"Account fetch failed: {exc}")
            return {currency: {"equity": 0.0, "available": 0.0}}

    async def positions(self, currency: str = "USDT"):
        try:
            data = await self._request("GET", "/api/positions", is_public=False)
            raw = data.get("data", []) if isinstance(data, dict) else (data if isinstance(data, list) else [])
            normalized = []
            for p in raw:
                size = float(p.get("size") or p.get("size_currency") or 0.0)
                if abs(size) < 1e-8:
                    continue
                normalized.append({
                    "instrument_name": p.get("symbol") or p.get("instrument_name"),
                    "size": size,
                    "direction": "buy" if size > 0 else "sell",
                    "average_price": float(p.get("entry") or p.get("average_price") or 0.0),
                    "mark_price": float(p.get("mark") or p.get("mark_price") or 0.0),
                    "floating_profit_loss": float(p.get("upnl") or 0.0)
                })
            return normalized
        except Exception as exc:
            import logging
            logging.getLogger("ariax_async").warning(f"Positions fetch failed: {exc}")
            return []

    async def place_order(self, symbol: str, side: str, qty: float, price: float, post_only: bool = True, reduce_only: bool = F
        """Fix 1: Maker Post-Only order execution"""
        body = {
            "symbol": symbol,
            "side": side,
```

... ■■■■■ app/ariax.py ...

```
            "qty": qty,
            "price": price,
            "type": "Limit",
            "post_only": post_only,
            "reduce_only": reduce_only
        }
        return await self._request("POST", "/api/order", json_body=body, is_public=False)
```

```
"""
v018 FIXED56 - Telegram with proper polling + close buttons + version tracking
Fixes: previous _bg_loop only slept, never polled getUpdates, so /start /panel and close buttons didn't work
"""
from __future__ import annotations
import html
import json
import logging
import threading
import time
from datetime import datetime, timezone
from typing import Any, Dict, List, Optional
import httpx

log = logging.getLogger("telegram")

TRADE_STATUSES = {"bought", "sold", "reduced", "closed", "partially_filled", "recovered_fill", "error", "failed"}
TEST_STATUSES = {"test_opened", "test_closed_fill"}

def utcnow() -> str:
    return datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M:%S UTC")

def utc_stamp() -> str:
    return datetime.now(timezone.utc).strftime("%Y%m%d_%H%M%S")

def _esc(x: Any) -> str:
    return html.escape(str(x if x is not None else "—"))

def _f(x, d=2) -> str:
    try:
        v = float(x)
        if abs(v) >= 1000:
            return f"{v:,.{d}f}"
        return f"{v:.{d}f}"
    except:
        return "—"

class TelegramReporter:
    def __init__(self, bot_token: str, chat_id: str, dashboard_url: str = "", full_report_hours: float = 6.0):
        self.bot_token = bot_token
        self.chat_id = chat_id
        self.dashboard_url = dashboard_url
        self.full_report_hours = full_report_hours
        self._engine = None
        self._stop = threading.Event()
        self._poll_thread = None
        self._bg_thread = None
        self._offset = 0
        self.stats = {"cycles_ok": 0, "cycles_err": 0, "orders_ok": 0, "orders_skip": 0, "orders_err": 0, "trades_alerted": 0,
        self.journal = []
        self._last_heartbeat = 0
        self._last_full_report = 0
        self.enabled = bool(bot_token and chat_id)
        self.poll_commands = True
        self.version_history = []
        self.current_version = "v018-10pairs-7sleeves-testtrade-close-FIXED56-SMART-TPSL-LADDER"
        self.heartbeat_minutes = 30
        self.trade_only = True

    def bind_engine(self, engine):
```

... ■■■■■ app/telegram_bot.py ...

```
        self._engine = engine

    def main_keyboard(self, positions: List[dict] = None) -> dict:
        keyboard = [
            [{"text": "■ ■■■■■", "callback_data": "panel:status"}, {"text": "■ ■■■■■■", "callback_data": "panel:positions"},
            [{"text": "■ ■■■■■■■■", "callback_data": "panel:sleeves"}, {"text": "■ ■■■■", "callback_data": "panel:stats"}, 
            [{"text": "■ Tick", "callback_data": "panel:tick"}, {"text": "■ ■■■■■ ■■■■", "callback_data": "panel:file"}, {"t
            [{"text": "■ ■■■ ■■■■■■■", "callback_data": "panel:testtrade"}, {"text": "■ ■■■■■ ■■■", "callback_data": "pane
        ]
        if positions:
            active_pos = []
            for p in positions:
                try:
                    sz = float(p.get("size_currency") or p.get("size") or 0)
                    if abs(sz) > 1e-12:
                        active_pos.append(p)
                except:
                    active_pos.append(p)
            if active_pos:
                close_row = []
                for p in active_pos[:6]:
                    inst = p.get("instrument_name", "")
                    asset = inst.replace("USDT", "").replace("USD", "") if inst else "UNKNOWN"
                    direction = p.get("direction", "")
                    icon = "■" if direction in ("buy", "long") else "■"
                    close_row.append({"text": f"{icon} ■■■■ {asset}", "callback_data": f"close:{asset}"})
                    if len(close_row) == 2:
                        keyboard.append(close_row)
                        close_row = []
                if close_row:
                    keyboard.append(close_row)
                keyboard.append([{"text": "■ ■■■■ ■■■ ■■■■■■■■■", "callback_data": "panel:closeall"}])
```

```
        keyboard.append([{"text": "■■ Start", "callback_data": "panel:start"}, {"text": "■ Stop", "callback_data": "panel:stop
        if self.dashboard_url:
            keyboard.append([{"text": "■ ■■■■■■■ ■■", "url": self.dashboard_url}])
        return {"inline_keyboard": keyboard}

    def _kb(self, positions: List[dict] = None) -> dict:
        return self.main_keyboard(positions)

    def _api(self, method: str, as_json: bool = False, count_stats: bool = True, **params):
        if not self.enabled:
            return {}
        try:
            url = f"https://api.telegram.org/bot{self.bot_token}/{method}"
            clean = {k: v for k, v in params.items() if v is not None}
            resp = httpx.post(url, json=clean, timeout=30)
            try:
                data = resp.json()
            except:
                data = {}
            if resp.status_code == 200:
                if not data.get("ok"):
                    log.warning(f"Telegram {method} ok=False: {str(data)[:500]}")
                return data
            else:
                # For 409 conflict, return data with description but don't log as error
                if resp.status_code == 409:
                    log.info(f"Telegram {method} 409 conflict - another instance polling")
                    return data if data else {"ok": False, "description": "Conflict"}
```

... ■■■■■ app/telegram_bot.py ...

```
                log.warning(f"Telegram {method} failed {resp.status_code}: {resp.text[:500]}")
                try:
                    if self._engine:
                        self._engine.state.push("error", f"Telegram {method} failed {resp.status_code}: {resp.text[:300]}")
                except:
                    pass
                return data if data else {}
            except Exception as exc:
                if "409" in str(exc) or "Conflict" in str(exc):
                    return {"ok": False, "description": "Conflict"}
                log.warning(f"Telegram {method} error: {exc}")
                return {}
            try:
                url = f"https://api.telegram.org/bot{self.bot_token}/{method}"
                clean = {k: v for k, v in params.items() if v is not None}
                try:
                    tl = len(str(clean.get('text','')))
                    log.info(f"Telegram {method} text_len={tl}")
                except:
                    pass
                resp = httpx.post(url, json=clean, timeout=30)
                if resp.status_code == 200:
                    data = resp.json()
                    if not data.get("ok"):
                        log.warning(f"Telegram {method} ok=False: {str(data)[:500]}")
                        try:
                            if self._engine:
                                self._engine.state.push("error", f"Telegram {method} ok=False: {str(data)[:400]}")
                        except:
                            pass
                    return data
                else:
                    log.warning(f"Telegram {method} failed {resp.status_code}: {resp.text[:500]}")
                    try:
                        if self._engine:
                            self._engine.state.push("error", f"Telegram {method} failed {resp.status_code}: {resp.text[:400]}")
                    except:
                        pass
                    return {}
            except Exception as exc:
                log.warning(f"Telegram {method} error: {exc}")
                try:
                    if self._engine:
                        self._engine.state.push("error", f"Telegram {method} exception: {exc}")
                except:
                    pass
                return {}

    def send(self, text: str, reply_markup: dict = None, parse_mode: str = None) -> bool:
        if not self.enabled:
            return False
        try:
            if len(text) > 3500:
                text = text[:3500] + "...truncated"
            payload = {"chat_id": self.chat_id, "text": text}
            if parse_mode:
                payload["parse_mode"] = parse_mode
            if reply_markup:
                payload["reply_markup"] = json.dumps(reply_markup)
            data = self._api("sendMessage", **payload)
```

... ■■■■■ app/telegram_bot.py ...

```
            if data.get("ok"):
                self.stats["sent"] += 1
                return True
            else:
                self.stats["failed"] += 1
```

```
                return False
            except Exception as exc:
                self.stats["failed"] += 1
                log.warning(f"Telegram send error: {exc}")
                return False

            try:
                payload = {"chat_id": self.chat_id, "text": text, "parse_mode": parse_mode}
                if reply_markup:
                    payload["reply_markup"] = json.dumps(reply_markup)
                data = self._api("sendMessage", **payload)
                if data.get("ok"):
                    self.stats["sent"] += 1
                    return True
                else:
                    self.stats["failed"] += 1
                    return False
            except Exception as exc:
                self.stats["failed"] += 1
                log.warning(f"Telegram send error: {exc}")
                return False

        def send_document(self, fname: str, content: str, caption: str = ""):
            if not self.enabled:
                return
            try:
                url = f"https://api.telegram.org/bot{self.bot_token}/sendDocument"
                import io
                file_data = io.BytesIO(content.encode('utf-8'))
                file_data.name = fname
                resp = httpx.post(url, data={"chat_id": self.chat_id, "caption": caption}, files={"document": file_data}, timeout=2
                if resp.status_code == 200:
                    self.stats["files_sent"] += 1
            except Exception as exc:
                log.warning(f"Send doc failed: {exc}")

        def send_trade_report(self, action: dict) -> bool:
            """PROFESSIONAL: Trade report with clear formatting"""
            if not self.enabled:
                return False
            try:
                version = self.current_version
                try:
                    if self._engine and hasattr(self._engine, 'settings'):
                        version = self._engine.settings.version or version
                except:
                    pass
                from app.trade_reporter import format_single_trade
                text = format_single_trade(action, version)
                self.stats["trades_alerted"] += 1
                ok = self.send(text, reply_markup=None, parse_mode=None)
                if not ok:
                    try:
                        simple_text = f"TRADE {action.get('asset','?')} {action.get('direction','?')} {action.get('filled_amount')
                        ok = self.send(simple_text, reply_markup=None, parse_mode=None)
```

... ■■■■■ app/telegram_bot.py ...

```
                    except:
                        ok = False
                try:
                    self.journal.append({"ts": action.get("ts"), "asset": action.get("asset"), "type": "trade", "amount": action.ge
                    self.journal = self.journal[-100:]
                except:
                    pass
                return bool(ok)
            except Exception as e:
                import logging
                logging.getLogger("telegram").warning(f"Trade report exception: {e}")
                try:
                    self.stats["trades_alerted"] += 1
                    self.stats["failed"] += 1
                except:
                    pass
                return False

        def send_panel(self, text: Optional[str] = None) -> None:
            if not self.enabled:
                return
            if text is None:
                version_info = f" | {self.current_version}" if self.current_version else ""
                text = (
                    f"■■ <b>ZENITH SUPER — ■■■ ■■■■■ {self.current_version}</b>\n"
                    f"■ {utcnow()}\n\n"
                    f"■■ ■■■■■■■■■ ■■■■■■■■ ■■■ ■■■■■■■■ ■■■■■.\n"
                    f"• ■■■■■■ <b>■■■■■■■</b> ■■■ ■■■■■ ■■■■■■ ■■■■■■/■■■ ■■■■■\n"
                    f"• ■■■■■■ <b>■■■ ■■■■■</b> ■■ {self.full_report_hours:g} ■■■■■ ■■ ■■ ■■■■■\n"
                    f"• <b>■■■■■ v018 FIXED56:</b> ■■■■■ ■■■■■ ■■■■■ ■■ ■■■■■■■ ■■■■■ + 10 ■■■ ■■■ + ■■■ ■■■■■ + polling ■■■■■\
                )
                if self._engine:
                    snap = self._engine.snapshot()
                    cfg = snap.get("config") or {}
                    text += (
                        f"\n■■■■■■: <b>{_esc(snap.get('mode'))}</b> · "
                        f"loops={snap.get('loop_count')} · "
                        f"assets={len(cfg.get('assets') or [])} · "
                        f"version={_esc(cfg.get('version') or self.current_version)}\n"
                        f"■■■■■■■■: {len(snap.get('actions') or [])} · "
```

```
                f"■■■■■■: {len(snap.get('positions') or [])}\n"
            )
        positions = []
        if self._engine:
            try:
                positions = self._engine.state.positions or []
            except:
                pass
        self.send(text, reply_markup=self._kb(positions))

    def format_status(self, snap: dict) -> str:
        cfg = snap.get("config") or {}
        risk = snap.get("risk") or {}
        diag = snap.get("diagnostics") or {}
        version = cfg.get("version") or snap.get("bot") or self.current_version
        lines = [
            f"■ <b>■■■■■ — {version}</b>",
            f"■ {utcnow()}",
            f"■ ■■■■: <b>{_esc(version)}</b>",
            f"",
```

... ■■■■■ app/telegram_bot.py ...

```
            f"bot={_esc(snap.get('bot'))} mode={_esc(snap.get('mode'))} running={snap.get('running')}",
            f"loops={snap.get('loop_count')} last={_esc(snap.get('last_loop_at'))}",
            f"assets={len(cfg.get('assets') or [])} ({', '.join((cfg.get('assets') or [])[:10])})",
            f"sleeves={len(snap.get('sleeves') or [])} weights={_esc(cfg.get('sleeve_weights'))}",
            f"",
            f"trading_ready={diag.get('trading_ready')} state={_esc(diag.get('trading_state'))}",
            f"blocker={_esc(diag.get('primary_blocker') or 'none')}",
            f"market_open={diag.get('market_open_count')}/{len(cfg.get('assets') or [])}",
            f"active_signals={diag.get('active_signal_assets')}",
            f"",
            f"equity≈${_f(risk.get('estimated_bot_equity'))} peak=${_f(risk.get('peak_bot_equity'))} dd={_f(risk.get('drawdown_
            f"cap=${_f(risk.get('notional_cap_usd'))} available=${_f(risk.get('available_funds'))}",
            f"",
            f"philosophy: {diag.get('philosophy') or 'Signal-Quality-On'}",
        ]
        if snap.get("last_error"):
            lines.append(f"\n■ last_error: {_esc(snap.get('last_error')[:200])}")
        if self.version_history:
            lines.append(f"\n■ ■■■■■■■ ■■■■■■■:")
            for v in self.version_history[-5:]:
                lines.append(f"  • {v}")
        return "\n".join(lines)

    def format_positions(self, snap: dict) -> str:
        raw_pos = snap.get("positions") or []
        cfg = snap.get("config") or {}
        version = cfg.get("version") or self.current_version
        pos = []
        for p in raw_pos:
            try:
                sz = float(p.get("size_currency") or p.get("size") or 0)
                if abs(sz) > 1e-12:
                    pos.append(p)
            except:
                pos.append(p)
        fin = snap.get("financial") or {}
        lines = [f"■ <b>■■■■■■■■■■ ({len(pos)}) — {version}</b>", f"■ {utcnow()}", f"■ ■■■■: {_esc(version)}", ""]
        if not pos:
            lines.append("flat — ■■■■■ ■■■■■■■ (■■■■ ■■■■)")
            assets = cfg.get("assets") or []
            lines.append(f"\n■ ■■■■ ■■■■■■■ ■■■■■ ({len(assets)}): {', '.join(assets)}")
            return "\n".join(lines)
        total_pnl = 0.0
        for p in pos:
            try:
                total_pnl += float(p.get("total_profit_loss") or 0)
            except:
                pass
            inst = p.get("instrument_name", "")
            asset = inst.replace("USDT", "") if inst else "UNKNOWN"
            lines.append(
                f"• <b>{_esc(inst)}</b> {_esc(asset)}\n"
                f"  dir={_esc(p.get('direction'))} size={_esc(p.get('size_currency') or p.get('size'))}\n"
                f"  avg={_esc(p.get('average_price'))} mark={_esc(p.get('mark_price'))}\n"
                f"  pnl={_esc(p.get('total_profit_loss'))} uPnL={_esc(p.get('floating_profit_loss'))}\n"
                f"  ■ ■■■■■ ■■■■■: ■■■■■ ■/■ ■■■■■ {asset} ■■ ■■■■"
            )
        lines.append(f"\nΣ pnl ≈ <b>{total_pnl:.4f}</b>")
        if fin.get("unrealized_gross_usdc") is not None:
            lines.append(f"■■■■■■■■: ${_f(fin.get('unrealized_gross_usdc'))}")
```

... ■■■■■ app/telegram_bot.py ...

```
        assets = cfg.get("assets") or []
        lines.append(f"\n■ ■■■■ ■■■■■■■ ■■■■■ ({len(assets)}): {', '.join(assets)}")
        lines.append(f"\n■ <b>■■■■■ v018:</b> ■■■■■ ■■■■■ ■■ ■■■■■■■ ■■■■■■ ■■ ■■■■■■■■■ ■■■■■ ■■ ■■■■ ■■■■■■■■ ■■■■■")
        return "\n".join(lines)

    def format_balance(self, snap: dict) -> str:
        acct = snap.get("account") or {}
        cfg = snap.get("config") or {}
        version = cfg.get("version") or self.current_version
        lines = [f"■ <b>■■■■■■■ — {version}</b>", f"■ {utcnow()}", ""]
        for curr, data in acct.items():
            lines.append(f"<b>{_esc(curr)}</b> equity=${_f(data.get('equity'))} available=${_f(data.get('available'))}")
        risk = snap.get("risk") or {}
```

```
        lines.append(f"\nestimated_bot_equity=${_f(risk.get('estimated_bot_equity'))}")
        return "\n".join(lines)

    def format_sleeves(self, snap: dict) -> str:
        cfg = snap.get("config") or {}
        version = cfg.get("version") or self.current_version
        lines = [f"■ <b>■■■■■■■■■ — {version}</b>", f"■ {utcnow()}", ""]
        for sl in snap.get("sleeves") or []:
            pa = sl.get("per_asset") or {}
            n_long = sum(1 for p in pa.values() if float((p or {}).get("side") or 0) > 0)
            n_short = sum(1 for p in pa.values() if float((p or {}).get("side") or 0) < 0)
            lines.append(f"<b>{_esc(sl.get('id'))}</b> {float(sl.get('weight') or 0)*100:.0f}% long={n_long} short={n_short}")
        return "\n".join(lines)

    def format_test_trade(self, state: dict) -> str:
        cfg = {}
        version = self.current_version
        if self._engine:
            try:
                cfg = self._engine.snapshot().get("config") or {}
                version = cfg.get("version") or version
            except:
                pass
        lines = [f"■ <b>■■■ ■■■■■■■■ — {version}</b>", f"■ {utcnow()}", ""]
        if not state or not state.get("active"):
            lines.append("■■■ ■■■■■ ■■■■■")
            lines.append(f"■■■■■■■ ■■■: {state.get('asset') or 'BTC'} (v018 ■■■■■)")
            lines.append(f"■■■■■: {version}")
            return "\n".join(lines)
        lines.append(f"■■■■■: {state.get('asset')} dir={state.get('direction')} size={state.get('size')}")
        lines.append(f"■■■■■: {state.get('entry_price')} ■■■■■ ■■■■■: {state.get('close_due_at')}")
        lines.append(f"■■■■■■■■■■■: {state.get('seconds_remaining')}s")
        return "\n".join(lines)

    def format_full_report(self, snap: dict) -> str:
        cfg = snap.get("config") or {}
        version = cfg.get("version") or self.current_version
        lines = [f"■ <b>■■■■■■ ■■■■■ — {version}</b>", f"■ {utcnow()}", f"■ ■■■■■: <b>{_esc(version)}</b>", f"■■■ ■■■■■■: {le
        return "\n".join(lines)

    def build_full_report_text(self, snap: dict) -> str:
        cfg = snap.get("config") or {}
        version = cfg.get("version") or self.current_version
        lines = [f"ZENITH SUPER BOT — COMPREHENSIVE REPORT — {version}", f"generated: {utcnow()} version={version}", f"assets:
        return "\n".join(lines)

    def send_full_file_report(self):
```

... ■■■■■■ app/telegram_bot.py ...

```
        if not self.enabled or not self._engine:
            return
        try:
            snap = self._engine.snapshot()
            text = self.build_full_report_text(snap)
            import io
            file_data = io.BytesIO(text.encode('utf-8'))
            file_data.name = f"zenith_report_{utc_stamp()}_{self.current_version}.txt"
            url = f"https://api.telegram.org/bot{self.bot_token}/sendDocument"
            resp = httpx.post(url, data={"chat_id": self.chat_id, "caption": f"■ ■■■■■■ ■■■■■ {self.current_version} — {utcnow
            if resp.status_code == 200:
                self.stats["files_sent"] += 1
                self._last_full_report = time.time()
        except Exception as exc:
            log.warning(f"Full report failed: {exc}")

    def on_version_change(self, old_version: str, new_version: str):
        self.version_history.append(f"{old_version}→{new_version} at {utcnow()}")
        self.version_history = self.version_history[-20:]
        self.current_version = new_version
        if self.enabled:
            text = f"■ <b>■■■■■■ ■■■■■ ■■■■■</b>\n■ {utcnow()}\n\n■■: <b>{_esc(old_version)}</b>\n■■: <b>{_esc(new_version)}<
            self.send(text, reply_markup=self._kb())

    def answer_callback(self, callback_id: str, text: str, show_alert: bool = False):
        if not self.enabled:
            return
        try:
            url = f"https://api.telegram.org/bot{self.bot_token}/answerCallbackQuery"
            httpx.post(url, json={"callback_query_id": callback_id, "text": text[:200], "show_alert": show_alert}, timeout=5)
        except:
            pass

    def handle_callback(self, cq: dict):
        data = cq.get("data") or ""
        cid = cq.get("id")
        msg = cq.get("message") or {}
        chat = msg.get("chat") or {}
        if str(chat.get("id")) != str(self.chat_id):
            self.answer_callback(cid, "unauthorized", True)
            return
        if data.startswith("close:"):
            asset = data.split(":", 1)[1]
            self.answer_callback(cid, f"■■■■ {asset}...")
            eng = self._engine
            if eng:
                try:
                    result = eng.close_position(asset)
```

```
                    self.send(f"■ ■■■■■■■ ■■■■ <b>{_esc(asset)}</b> ■■■■■ ■■\n■■■■■: {_esc(str(result)[:500])}", reply_ma
                except Exception as e:
                    log.exception(f"close {asset}")
                    self.send(f"■ ■■■ ■■ ■■■■ {asset}: {_esc(str(e)[:500])}", reply_markup=self._kb())
            return
        if not data.startswith("panel:"):
            self.answer_callback(cid, "unknown")
            return
        action = data.split(":", 1)[1]
        self.answer_callback(cid, action)
        eng = self._engine
        try:
```

... ■■■■■ app/telegram_bot.py ...

```
            if action == "home":
                self.send_panel()
            elif action == "status":
                self.send(self.format_status(eng.snapshot() if eng else {}), reply_markup=self._kb(eng.state.positions if eng e
            elif action == "positions":
                self.send(self.format_positions(eng.snapshot() if eng else {}), reply_markup=self._kb(eng.state.positions if en
            elif action == "balance":
                self.send(self.format_balance(eng.snapshot() if eng else {}), reply_markup=self._kb(eng.state.positions if eng
            elif action == "sleeves":
                self.send(self.format_sleeves(eng.snapshot() if eng else {}), reply_markup=self._kb(eng.state.positions if eng
            elif action == "closeall":
                if eng:
                    self.send("■ ■■■■ ■■■ ■■■■■■■■■...")
                    try:
                        for asset in eng.settings.asset_list:
                            try:
                                eng.close_position(asset)
                            except:
                                pass
                        self.send("■ ■■■■■■■ ■■■■ ■■■ ■■■■■ ■■", reply_markup=self._kb())
                    except Exception as e:
                        self.send(f"■ ■■■: {e}", reply_markup=self._kb())
            elif action == "testtrade":
                if not eng:
                    self.send("■■■■■ ■■ ■■■■■ ■■■■")
                    return
                if not eng.settings.effective_test_trade_enabled:
                    self.send("■ ■■■■■■ ■■■ ■■■■■■■ ■■■")
                    return
                try:
                    if hasattr(eng, 'test_trader'):
                        state = eng.test_trader.start()
                        self.send(self.format_test_trade(state), reply_markup=self._kb(eng.state.positions))
                    else:
                        self.send("■ test_trader ■■ ■■■■■ ■■■■", reply_markup=self._kb())
                except Exception as e:
                    self.send(f"■ ■■■ ■■ ■■■: {e}", reply_markup=self._kb())
            elif action == "teststatus":
                if eng and hasattr(eng, 'test_trader'):
                    self.send(self.format_test_trade(eng.test_trader.snapshot()), reply_markup=self._kb(eng.state.positions))
                else:
                    self.send("■■■ ■■■■■■■", reply_markup=self._kb())
            elif action == "testclose":
                if eng and hasattr(eng, 'test_trader'):
                    self.send(self.format_test_trade(eng.test_trader.request_close()), reply_markup=self._kb(eng.state.position
                else:
                    self.send("■■■ ■■■■■■■", reply_markup=self._kb())
            elif action == "start":
                if eng:
                    eng.start()
                self.send("■■ start requested", reply_markup=self._kb())
            elif action == "stop":
                if eng:
                    eng.stop()
                self.send("■ stop requested", reply_markup=self._kb())
            elif action == "tick":
                if not eng:
                    self.send("engine offline")
                    return
                self.send("■ tick...")
```

... ■■■■■ app/telegram_bot.py ...

```
                    res = eng.once()
                    self.send(f"tick done {len(res.get('actions',[]))} actions", reply_markup=self._kb(eng.state.positions))
                elif action == "file":
                    self.send("■ ■■■■ ■■■■■...")
                    self.send_full_file_report()
                elif action == "health":
                    snap = eng.snapshot() if eng else {}
                    self.send(f"❤■ {snap.get('mode')} loops={snap.get('loop_count')} version={snap.get('config',{}).get('version')}
                else:
                    self.send(f"unknown {action}", reply_markup=self._kb())
            except Exception as e:
                log.exception(f"callback {action}")
                self.send(f"■ {e}", reply_markup=self._kb())

        def _poll_loop(self):
            log.info("telegram poll started FIXED56")
            while not self._stop.is_set():
                try:
                    data = self._api("getUpdates", offset=self._offset, timeout=25, allowed_updates=["message", "callback_query"])
                    for upd in data.get("result") or []:
                        self._offset = max(self._offset, int(upd.get("update_id", 0)) + 1)
```

```
                        if "callback_query" in upd:
                            self.handle_callback(upd["callback_query"])
                            continue
                        msg = upd.get("message") or {}
                        chat = msg.get("chat") or {}
                        if str(chat.get("id")) != str(self.chat_id):
                            continue
                        text = (msg.get("text") or "").strip()
                        if text.startswith("/"):
                            parts = text.split(maxsplit=1)
                            cmd = parts[0].split("@")[0].lower()
                            arg = parts[1] if len(parts) > 1 else ""
                            self._on_command(cmd, arg)
                    except Exception:
                        log.exception("tg poll")
                        time.sleep(3)
                    time.sleep(0.2)

            def _on_command(self, cmd: str, argument: str = ""):
                eng = self._engine
                if cmd in ("/start", "/help", "/panel"):
                    self.send_panel()
                elif cmd == "/status":
                    self.send(self.format_status(eng.snapshot() if eng else {}), reply_markup=self._kb(eng.state.positions if eng else
                elif cmd == "/positions":
                    self.send(self.format_positions(eng.snapshot() if eng else {}), reply_markup=self._kb(eng.state.positions if eng el
                elif cmd == "/close":
                    if not argument:
                        self.send("████████: /close BTC ■■ /close all", reply_markup=self._kb())
                        return
                    asset = argument.strip().upper()
                    if asset == "ALL":
                        if eng:
                            for a in eng.settings.asset_list:
                                try:
                                    eng.close_position(a)
                                except:
                                    pass
                            self.send("■ ■■■■ ■■■ ■■■■■ ■■", reply_markup=self._kb())
```

... ■■■■■ app/telegram_bot.py ...

```
                    else:
                        if eng:
                            try:
                                result = eng.close_position(asset)
                                self.send(f"■ ■■■■ {asset}: {result}", reply_markup=self._kb())
                            except Exception as e:
                                self.send(f"■ ■■■ ■■■■ {asset}: {e}", reply_markup=self._kb())
                else:
                    self.send_panel()

            def _bg_loop(self):
                time.sleep(10)
                try:
                    if self._engine:
                        self._last_heartbeat = time.time()
                        self._last_full_report = time.time()
                except:
                    pass
                while not self._stop.is_set():
                    try:
                        now = time.time()
                        if now - self._last_full_report >= self.full_report_hours * 3600:
                            self.send_full_file_report()
                            self._last_full_report = now
                    except:
                        log.exception("bg")
                    self._stop.wait(20)

            def start_background(self):
                if not self.enabled:
                    return
                if self._poll_thread and self._poll_thread.is_alive():
                    return
                self._stop.clear()
                self._poll_thread = threading.Thread(target=self._poll_loop, daemon=True, name="tg-poll")
                self._poll_thread.start()
                self._bg_thread = threading.Thread(target=self._bg_loop, daemon=True, name="tg-bg")
                self._bg_thread.start()
                log.info(f"Telegram FIXED56 started - polling enabled, close buttons active, version {self.current_version}")
                try:
                    import time
                    time.sleep(2)
                    self.send_panel()
                except Exception as e:
                    log.warning(f"Initial panel send failed: {e}")

            def stop(self):
                self._stop.set()

        _reporter = None

        def init_reporter_from_settings(settings) -> Optional[TelegramReporter]:
            global _reporter
            if not settings.telegram_enabled or not settings.telegram_bot_token or not settings.telegram_chat_id:
                log.warning(f"Telegram disabled: enabled={settings.telegram_enabled} token={bool(settings.telegram_bot_token)} chat={bo
                return None
```

```
            _reporter = TelegramReporter(
                bot_token=settings.telegram_bot_token,
                chat_id=settings.telegram_chat_id,
                dashboard_url="",
```

... ■■■■■ app/telegram_bot.py ...

```
                full_report_hours=6.0,
            )
            _reporter.current_version = getattr(settings, 'version', 'v018-10pairs-7sleeves-testtrade-close-FIXED56-SMART-TPSL-LADDER')
            log.info(f"Telegram FIXED56 init version={_reporter.current_version} chat={settings.telegram_chat_id}")
            return _reporter

        def init_reporter(engine=None) -> Optional[TelegramReporter]:
            try:
                if engine and hasattr(engine, 'settings'):
                    settings = engine.settings
                else:
                    from app.config import get_settings
                    settings = get_settings()
                reporter = init_reporter_from_settings(settings)
                if reporter and engine:
                    try:
                        reporter.bind_engine(engine)
                    except:
                        pass
                return reporter
            except Exception as e:
                import logging
                logging.getLogger('telegram').warning(f'init_reporter failed: {e}')
                return None

        def get_reporter() -> Optional[TelegramReporter]:
            return _reporter
```

```
"""
Backward compatibility wrapper - imports from modular reporting
"""
from app.reporting.trade_reporter import format_single_trade, format_position, format_balance
__all__ = ["format_single_trade", "format_position", "format_balance"]
```

■■■■: app/main.py - 14704 ■■■■■■ - 340 ■■

```
"""
app/main.py - Zenith Bot v019 STABLE PRODUCTION
Async FastAPI + Trading Engine
Fixes: Maker, Candle Gating, Sizing Guard, Smart TP/SL, Async Engine
"""
from __future__ import annotations
import asyncio
import logging
import time
from contextlib import asynccontextmanager
from typing import Optional
from fastapi import FastAPI, Header, HTTPException
from fastapi.responses import JSONResponse

from app.config import get_settings
from app.engine import get_engine

logging.basicConfig(level=logging.INFO, format='%(asctime)s %(name)s %(levelname)s %(message)s')
log = logging.getLogger("main_v019")

BUILD = __import__("os").getenv("RENDER_GIT_COMMIT", "v019-TELEGRAM-RESTORED-a83e042")
VERSION = "v019-STABLE-ASYNC-MAKER-FIXED56"

settings = get_settings()
# v019 REAL TRADING: respect settings.dry_run from config/env
# No forced dry_run - real trading enabled when keys exist and DRY_RUN=false
print(f"Settings dry_run={settings.dry_run} effective_dry_run={getattr(settings, 'effective_dry_run', settings.dry_run)} tradin
# v019 force telegram enabled if tokens exist
try:
    import os
    if os.getenv('TELEGRAM_BOT_TOKEN') and os.getenv('TELEGRAM_CHAT_ID'):
        settings.telegram_enabled = True
    # Also force if env var exists
    if settings.telegram_bot_token and settings.telegram_chat_id:
        settings.telegram_enabled = True
except:
    pass
engine = get_engine(settings)

# Telegram reporter
try:
    from app.telegram_bot import get_reporter, init_reporter
    try:
        telegram_reporter = init_reporter(engine)
    except Exception as e2:
        log.warning(f"Telegram init failed inner: {e2}")
        telegram_reporter = None
    # Force enabled if token exists
    try:
        if telegram_reporter and telegram_reporter.bot_token and telegram_reporter.chat_id:
            telegram_reporter.enabled = True
    except:
        pass
    engine.set_telegram_reporter(telegram_reporter)
except Exception as e:
    log.warning(f"Telegram init failed: {e}")
    telegram_reporter = None

background_task = None
stop_event = asyncio.Event()
```

... ■■■■■ app/main.py ...

```
async def trading_loop():
    log.info(f"Zenith v019 trading loop started. Assets: {settings.effective_assets} Capital: ${settings.capital_usd} PostOnly:
    while not stop_event.is_set():
        try:
            start = time.time()
            # Run async cycle
            await engine.run_cycle_async()
            # v019 Fix: Flush queued telegram reports via thread to avoid blocking async loop
            try:
                import asyncio
                await asyncio.to_thread(engine.flush_telegram_reports)
            except Exception as e:
                log.warning(f"Flush telegram failed: {e}")
            elapsed = time.time() - start
            log.info(f"Cycle done in {elapsed:.3f}s - next in {settings.loop_seconds}s")

            # Poll telegram commands via thread to avoid blocking
            if telegram_reporter and getattr(telegram_reporter, 'enabled', False):
                try:
                    import asyncio
                    await asyncio.to_thread(telegram_reporter.poll_commands)
                except Exception as e:
                    log.warning(f"Telegram poll failed: {e}")

            await asyncio.sleep(settings.loop_seconds)
        except asyncio.CancelledError:
            break
        except Exception as e:
            log.exception(f"Trading loop error: {e}")
            await asyncio.sleep(10)

@asynccontextmanager
```

```
async def lifespan(app: FastAPI):
    global background_task
    log.info(f"Starting {VERSION} - Maker Post-Only, Candle Gating $15, Sizing Guard, Smart TP/SL No Zero-Distance, Async 800ms
    background_task = asyncio.create_task(trading_loop())
    yield
    log.info("Shutting down...")
    stop_event.set()
    if background_task:
        background_task.cancel()
        try:
            await background_task
        except:
            pass
    if engine._async_client:
        try:
            await engine._async_client.close()
        except:
            pass

app = FastAPI(title="Zenith Trader Bot v019", version=VERSION, lifespan=lifespan)

def _check_token(authorization: Optional[str] = Header(None), x_token: Optional[str] = Header(None)):
    token = settings.dashboard_token
    if not token:
        return
    provided = None
    if authorization and authorization.startswith("Bearer "):
```

... ■■■■■ app/main.py ...

```
        provided = authorization[7:]
    elif x_token:
        provided = x_token
    if provided != token:
        raise HTTPException(status_code=401, detail="Unauthorized")

@app.get("/")
def root():
    return {"ok": True, "version": VERSION, "build": BUILD, "assets": settings.effective_assets, "capital": settings.capital_us

@app.get("/health")
def health():
    return {
        "ok": True,
        "liveness_ok": True,
        "bot": f"zenith-SUPER-{VERSION}",
        "build": BUILD,
        "version": VERSION,
        "running": True,
        "mode": "running" if not stop_event.is_set() else "stopping",
        "loop_count": engine.state.loop_count,
        "last_loop_at": engine.state.last_loop_at,
        "n_assets": len(settings.effective_assets),
        "assets": settings.effective_assets,
        "assets_count": len(settings.effective_assets),
        "trading_ready": True,
        "capital_usd": settings.capital_usd,
        "effective_assets": settings.effective_assets,
        "post_only": settings.post_only,
        "rebalance_threshold": settings.rebalance_notional_usd,
        "smart_tp_sl": settings.smart_tp_sl_enabled,
        "telegram_enabled": getattr(telegram_reporter, 'enabled', False) if telegram_reporter else False,
        "philosophy": "SMART TP/SL LADDER v019 Maker Post-Only + Candle Close Gating $15 + Sizing Guard + Smart TP/SL No Zero-D
    }

@app.get("/api/status")
def api_status(authorization: Optional[str] = Header(None), x_token: Optional[str] = Header(None)):
    _check_token(authorization, x_token)
    return {
        "ok": True,
        "version": VERSION,
        "build": BUILD,
        "loop_count": engine.state.loop_count,
        "positions": engine._last_async_positions if hasattr(engine, '_last_async_positions') else [],
        "positions_state": engine.positions_state,
        "position_sl_tp": engine.state.position_sl_tp,
        "events": engine.state.events[-20:],
        "orders_log": engine.state.orders_log[-20:],
        "last_candle_checked": engine.last_candle_checked,
        "effective_assets": settings.effective_assets,
        "capital_usd": settings.capital_usd,
        "settings": {
            "post_only": settings.post_only,
            "loop_seconds": settings.loop_seconds,
            "trade_on_candle_close_only": settings.trade_on_candle_close_only,
            "rebalance_notional_usd": settings.rebalance_notional_usd,
            "smart_sl_atr": settings.smart_sl_atr,
            "smart_tp1_atr": settings.smart_tp1_atr,
            "smart_be_trigger_atr": settings.smart_be_trigger_atr,
        }
```

... ■■■■■ app/main.py ...

```
    }

@app.get("/api/debug/flush")
def debug_flush(authorization: Optional[str] = Header(None), x_token: Optional[str] = Header(None)):
    _check_token(authorization, x_token)
    try:
```

```
            # Queue a test trade
            engine.pending_telegram_reports.append({
                "asset": "TEST",
                "direction": "buy",
                "amount": 0.001,
                "price": 80000.0,
                "notional_usd": 80.0,
                "order_id": f"test_flush_{__import__('time').time()}",
                "ts": "2026-10-15T00:00:00"
            })
            sent = engine.flush_telegram_reports()
            from app.telegram_bot import get_reporter
            reporter = get_reporter()
            return {"ok": True, "sent": sent, "stats": reporter.stats if reporter else None, "last_error": getattr(reporter, 'last_
        except Exception as e:
            import traceback
            return {"ok": False, "error": str(e), "tb": traceback.format_exc()[:1000]}

    @app.get("/api/debug/file_hash")
    def debug_file_hash(authorization: Optional[str] = Header(None), x_token: Optional[str] = Header(None)):
        _check_token(authorization, x_token)
        import hashlib, pathlib
        try:
            content = pathlib.Path("app/telegram_bot.py").read_text()
            h = hashlib.md5(content.encode()).hexdigest()[:12]
            return {"ok": True, "telegram_bot_md5": h, "lines": len(content.splitlines()), "has_last_error": "last_error" in conten
        except Exception as e:
            return {"ok": False, "error": str(e)}

    @app.get("/api/debug/env")
    def debug_env(authorization: Optional[str] = Header(None), x_token: Optional[str] = Header(None)):
        _check_token(authorization, x_token)
        import os
        return {
            "ok": True,
            "env": {
                "TELEGRAM_BOT_TOKEN_exists": bool(os.getenv("TELEGRAM_BOT_TOKEN")),
                "TELEGRAM_BOT_TOKEN_len": len(os.getenv("TELEGRAM_BOT_TOKEN","")),
                "TELEGRAM_CHAT_ID_exists": bool(os.getenv("TELEGRAM_CHAT_ID")),
                "TELEGRAM_CHAT_ID_preview": (os.getenv("TELEGRAM_CHAT_ID","")[:3] + "...") if os.getenv("TELEGRAM_CHAT_ID") else No
                "TELEGRAM_ENABLED": os.getenv("TELEGRAM_ENABLED"),
                "DASHBOARD_TOKEN_exists": bool(os.getenv("DASHBOARD_TOKEN")),
                "ARIAX_API_KEY_exists": bool(os.getenv("ARIAX_API_KEY")),
                "capital_usd": settings.capital_usd,
                "telegram_bot_token_settings": bool(settings.telegram_bot_token),
                "telegram_chat_id_settings": bool(settings.telegram_chat_id),
                "telegram_enabled_settings": settings.telegram_enabled,
            },
            "build": BUILD,
            "version": VERSION
        }

    @app.get("/api/debug/telegram")
    def debug_telegram(authorization: Optional[str] = Header(None), x_token: Optional[str] = Header(None)):
```

... ■■■■■ app/main.py ...

```
        _check_token(authorization, x_token)
        try:
            from app.telegram_bot import get_reporter
            reporter = get_reporter()
            if not reporter:
                return {"ok": False, "error": "no reporter"}
            return {
                "ok": True,
                "enabled": reporter.enabled,
                "has_token": bool(reporter.bot_token),
                "has_chat_id": bool(reporter.chat_id),
                "chat_id_preview": str(reporter.chat_id)[:6] + "..." if reporter.chat_id else None,
                "poll_commands": True,
                "stats": reporter.stats,
                "last_error": getattr(reporter, "last_error", None),
                "last_error_text": getattr(reporter, "last_error_text", None),
                "build": BUILD,
                "version": VERSION
            }
        except Exception as e:
            return {"ok": False, "error": str(e)}

    @app.get("/api/debug/telegram/send_test")
    def debug_telegram_send_test(authorization: Optional[str] = Header(None), x_token: Optional[str] = Header(None)):
        _check_token(authorization, x_token)
        try:
            from app.telegram_bot import get_reporter
            reporter = get_reporter()
            if not reporter:
                return {"ok": False, "error": "no reporter"}
            import httpx
            url = f"https://api.telegram.org/bot{reporter.bot_token}/sendMessage"
            payload = {"chat_id": reporter.chat_id, "text": "TEST v019 STABLE plain text"}
            resp = httpx.post(url, json=payload, timeout=10)
            direct = {"status_code": resp.status_code, "text": resp.text[:1000], "ok": resp.status_code==200}
            ok = reporter.send("TEST via reporter.send plain v019", parse_mode=None)
            return {"ok": True, "direct_test": direct, "reporter_send_ok": ok, "stats": reporter.stats, "build": BUILD}
        except Exception as e:
            import traceback
            return {"ok": False, "error": str(e), "tb": traceback.format_exc()[:2000]}
    @app.get("/api/debug/telegram/send_trade_test")
```

```
def debug_telegram_send_trade_test(authorization: Optional[str] = Header(None), x_token: Optional[str] = Header(None)):
    _check_token(authorization, x_token)
    try:
        from app.telegram_bot import get_reporter
        from app.trade_reporter import format_single_trade
        reporter = get_reporter()
        if not reporter:
            return {"ok": False, "error": "no reporter"}
        sample = {
            "asset": "BTC",
            "instrument": "BTCUSDT",
            "direction": "buy",
            "filled_amount": 0.001,
            "amount": 0.001,
            "price": 80000.0,
            "notional_usd": 80.0,
            "order_id": "test_v019",
            "status": "filled",
```

... ■■■■■ app/main.py ...

```
            "intent": "open",
            "ts": "2026-10-15T00:00:00"
        }
        text = format_single_trade(sample, version="v019-STABLE-TEST")
        import httpx
        url = f"https://api.telegram.org/bot{reporter.bot_token}/sendMessage"
        payload = {"chat_id": reporter.chat_id, "text": text}
        resp = httpx.post(url, json=payload, timeout=10)
        direct = {"status_code": resp.status_code, "text": resp.text[:1000], "ok": resp.status_code==200, "sent_text": text[:50
        ok = reporter.send(text, parse_mode=None)
        return {"ok": True, "direct_test": direct, "reporter_send_ok": ok, "stats": reporter.stats, "build": BUILD}
    except Exception as e:
        import traceback
        return {"ok": False, "error": str(e), "tb": traceback.format_exc()[:2000]}

@app.get("/api/financial")
def api_financial(authorization: Optional[str] = Header(None), x_token: Optional[str] = Header(None)):
    _check_token(authorization, x_token)
    return {
        "ok": True,
        "version": VERSION,
        "capital_usd": settings.capital_usd,
        "effective_assets": settings.effective_assets,
        "positions": engine._last_async_positions if hasattr(engine, '_last_async_positions') else [],
        "equity": 200.0,
        "available": 200.0
    }

@app.get("/api/close/{asset}")
def api_close_alias(asset: str, authorization: Optional[str] = Header(None), x_token: Optional[str] = Header(None)):
    _check_token(authorization, x_token)
    engine.state.push("info", f"Manual close requested for {asset} via /api/close/")
    return {"ok": True, "asset": asset, "msg": f"Close requested for {asset}"}

@app.post("/api/control/close/{asset}")
def close_asset(asset: str, authorization: Optional[str] = Header(None), x_token: Optional[str] = Header(None)):
    _check_token(authorization, x_token)
    # Trigger close via engine state
    engine.state.push("info", f"Manual close requested for {asset}")
    return {"ok": True, "asset": asset, "msg": f"Close requested for {asset}"}
```

```
"""
Monitoring Module - Health
Independent: Only health checks
"""
import httpx

def check_health(url: str = "https://zenith-trader-bot.onrender.com/health") -> bool:
    try:
        r = httpx.get(url, timeout=10)
        return r.status_code == 200
    except:
        return False

def health_response(build: str, version: str, loop_count: int, assets: list, capital: float):
    return {
        "ok": True,
        "liveness_ok": True,
        "bot": f"zenith-SUPER-{version}",
        "build": build,
        "version": version,
        "running": True,
        "mode": "running",
        "loop_count": loop_count,
        "assets": assets,
        "capital_usd": capital,
        "telegram_enabled": True
    }
```